ETH zürich

Master's Thesis

Network Security Group, Department of Computer Science, ETH Zurich

Intent-Driven Network Control in SCION

From Single-Path Selection to Joint Application-Multipath Co-Optimization

by Octave Charrin

Spring 2026

ETH student ID:	24-937-385
E-mail address:	ocharrin@ethz.ch
Supervisors:	Prof. Dr. Adrian Perrig Liwen Xu
Date of submission:	August 25, 2026

Abstract

The Internet forwards packets without regard for why they were sent, so the network optimizes a target that rarely matches what an application actually values. Path-aware architectures such as SCION change this by letting endpoints choose among multiple end-to-end paths, placing the path decision in the hands of the one party that knows the application's intent. This thesis studies how to make that choice well: how to select the best path – or the best combination of paths – given both the application's intent and the current state of the network.

We take a single idea from recent congestion-control work, that application intent should be a first-class input which reshapes a network mechanism, and push it into two settings. First, we turn a fixed-objective DQN path selector for SCION into an intent-conditioned one, combining a permutation-equivariant scoring architecture that handles a variable, changing set of candidate paths with an intent vector supplied to the per-path comparison itself; a single policy then serves a range of application intents without retraining, including intents it never trained on. We show, analytically and empirically, that *where* the conditioning enters decides whether it has any effect at all: a signal reaching only the terms shared by every candidate cancels under the comparison that selects a path, however much the policy is trained on it. In this environment the axis along which intents genuinely conflict is goodput against latency – loss is near zero on 99% of selections – so intent alignment is demonstrated there. Second, we ask what happens when several paths are used at once and the application is optimized alongside the network: a two-timescale hierarchical pair of reinforcement-learning agents co-optimizes a real-time video call's encoding bitrate and its per-path traffic split over Multipath QUIC. Here the central finding is that the value of learned multipath scheduling depends sharply on network volatility – nearly negligible when paths are stable, but decisive under path churn and congestion bursts, where controlling the application alone collapses. Together the two parts argue that on a path-aware Internet, application intent should steer every controllable decision between the application and the wire, and they chart what remains to make such control deployable.

Acknowledgements

[TODO: *Personal acknowledgements to be written before submission.*] I would like to thank my supervisor Liwen Xu for his guidance throughout this thesis, and Prof. Dr. Adrian Perrig for the opportunity to carry out this work in the Network Security Group at ETH Zurich. I am grateful to the members of the group for their feedback, and to the authors of the SCION DQN path-selection and Mortise systems, whose work this thesis builds upon. **[TODO:** *add thanks to family, friends, and anyone else who supported the work.*]

Contents

1 Introduction

The Internet was designed to deliver packets, not experiences. A router forwards a datagram toward its destination without any notion of *why* the datagram was sent: whether it carries a frame of an interactive video call that will be worthless if it arrives late, a byte of a background software update that only cares about aggregate throughput, or a control message whose loss would be catastrophic. For decades this deliberate ignorance – the end-to-end principle – was a feature. Yet as applications have become more diverse and more demanding, the mismatch between what the network optimizes and what the application actually values has grown into one of the central inefficiencies of modern networking.

Path-aware Internet architectures such as SCION [13] change the terms of this problem. In SCION, endpoints discover *multiple* end-to-end paths to a destination and choose among them, rather than deferring blindly to whatever route BGP happens to install. This turns path selection from a network-internal decision into an application-facing one: for the first time, the entity that knows the application's intent – the endpoint – also holds the lever that determines how traffic is carried. The fundamental question this thesis addresses is:

> *How should an endpoint choose the best path – or the best* combination *of paths – given both the intention of the application and the current state of the network?*

[TODO: *the three headings that follow ("The Research Challenge", "Key Insights and Approach", "Guiding Questions") are starred, so they carry no number and do not appear in the table of contents – which currently jumps from Chapter 1 straight to §1.1 Contributions. Either promote them to numbered sections, matching the 1.1 Motivation / 1.2 Contributions / 1.3 Organization structure the outline specifies, or demote them to* `\paragraph` *so the chapter reads as continuous prose.***]**

The Research Challenge

Answering this question is hard for three compounding reasons:

1. **"Best" is not a fixed target:** A path optimal for a bulk transfer (high goodput, tolerant of latency) is a poor choice for a video call (which prizes low latency and jitter). Hand-tuned objectives fail across diverse workloads, and training a separate reinforcement learning (RL) policy for every conceivable objective does not scale.

2. **The action space is dynamic and unbounded:** The set of candidate paths differs across source–destination pairs and fluctuates over time as links appear, fail, congest, or recover. Standard policies assuming a fixed, enumerated action set cannot cope.

3. **State is partially and expensively observable:** Fresh per-path measurements incur active probing overhead. A practical policy must decide well while measuring little.

Key Insights and Approach

This thesis addresses these challenges through a unified principle inspired by recent transport-layer auto-tuning. The Mortise system [16] shows that network mechanisms can be steered by treating application *intent* as an explicit, dynamic input rather than a constant compiled into an algorithm. We transplant this principle across two sequential operational control layers:

1. Motivating Foundation: Intent-Aware Single-Path Selection. We first investigate how application intent can modulate discrete path choice for general network flows. Building on the DQN path selector of Bourak et al. [3], we introduce a permutation-equivariant *path-scoring* architecture that supports variable candidate path sets. Crucially, we condition the agent on an explicit vector of application intent. We unveil a key theoretical insight: *in comparison-based selection, a conditioning signal that reaches only the terms shared by every candidate cancels under the argmax and cannot change the chosen path, no matter how the policy is trained*. Routing the intent instead into the *per-candidate* comparison lets a single policy serve diverse application intents without retraining, and interpolate to intents it never trained on. However, this study also exposes a physical limitation: when congestion surges, even the best single path reaches an architectural performance ceiling.

2. Joint Application–Network Optimization over Multipath. Driven by the limitations of single-path selection, we expand our framework to co-optimize both the application and the network simultaneously. Leveraging SCION's native path multiplicity and Multipath-QUIC [5] transport, we address interactive real-time video streaming by controlling both *what* the application produces (the video encoder's target bitrate) and *how* the network carries it (the continuous per-path traffic split). We design a two-timescale hierarchical reinforcement-learning system comprising a slow application agent for bitrate adaptation and a fast transport agent for path scheduling. This closes the loop from high-level intent down to application generation. Here, we uncover a crucial operational insight: *the value of learned multipath traffic splitting is a direct function of network volatility* – it offers marginal gains in static networks, but becomes decisive under path churn and congestion bursts where application-only adaptation collapses.

Guiding Questions

Throughout this investigation, four central puzzles guide the reader:

- *Can a single RL policy realize a range of application intents without retraining, and where must the intent enter the network for it to change the policy's choice at all?*
- *Where are the exact operational boundaries where single-path selection falls short?*
- *When is it worth splitting traffic across multiple paths, and does joint application–network co-optimization strictly beat optimizing either layer in isolation?*
- *How do these learned controllers perform regarding probing overhead, real-time inference latency, and deployability?*

1.1 Contributions

This thesis makes the following explicit, verifiable contributions:

1. **An intent-conditioned single-path selection framework for SCION:** We extend the DQN path selector of Bourak et al. [3] with a permutation-equivariant path-scoring network and per-path intent conditioning, enabling a single policy to realize a range of application intents without per-objective retraining, and to serve intents it never trained on *zero-shot*: behavior interpolates monotonically between trained objectives (Spearman $\rho = -1.000$), including on source–destination pairs never seen in training (Chapter 4).
2. **An analysis of where intent conditioning must enter:** We show analytically that a conditioning signal reaching only the terms shared by every candidate cannot change a comparison-based selection, and confirm empirically, over five training seeds with disjoint confidence intervals, that such a policy re-ranks roughly three times less than one whose conditioning reaches the per-candidate comparison (Chapter 4).
3. **Verifiable probing reduction at near-oracle quality:** We demonstrate that our learned single-path selector comes within 0.25% of a per-context reward oracle on every evaluated intent and outperforms every heuristic baseline on three of four, while cutting probing cost by roughly 40× against exhaustive bandwidth measurement and probing less than every heuristic in absolute terms (Chapter 4).
4. **A hierarchical system for joint application–network co-optimization:** We design and implement a two-timescale hierarchical SAC framework for real-time video over Multipath QUIC, co-optimizing target bitrates and continuous path splits across NS-3 packet simulation and analytical testbeds (Chapter 5).

5. **Characterization of the volatility driver for multipath control:** We demonstrate empirically that joint multipath control provides dramatic QoE improvements over application-only baselines specifically under high network volatility, path churn, and transient congestion bursts (Chapter 5 and §6.2).

1.2 Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews technical foundations, including SCION, real-time video transport, deep RL machinery, and goal-conditioned policies. Chapter 3 formalizes the decision problems, system models, desired properties, and trust assumptions.

Chapter 4 presents our study on intent-conditioned single-path selection, detailing its scoring architecture, the conditioning result and its proof, and the performance limits it exposes. Chapter 5 expands into joint application–network optimization over multipath, detailing the two-timescale SAC architecture and evaluation across dynamic network regimes.

Chapter 6 unifies the findings, analyzing overhead, scalability, generalization, and deployment considerations (Sections 6.1 and 6.2). Chapter 7 places our work within the broader research landscape, and Chapter 8 concludes.

2 Background & Technical Foundations

This chapter reviews the systems, control mechanisms, and machine learning techniques upon which this thesis builds. Section 2.1 introduces the SCION path-aware Internet architecture and the path-selection opportunities it creates. Section 2.2 covers quality-of-experience metrics for real-time video, adaptive bitrate control, and multipath transport mechanisms. Section 2.3 recalls the core reinforcement learning (RL) machinery: Markov decision processes, deep Q-networks with standard refinements, soft actor-critic, and permutation-equivariant scoring architectures for dynamic action spaces. Finally, Section 2.4 details goal-conditioned policy representations and the goal-conditioning machinery that lets one policy serve many objectives.

2.1 The SCION Path-Aware Architecture

SCION [13] is a next-generation, path-aware Internet architecture designed for high availability and transparency of the forwarding path. Its defining departure from today's Internet is that *end hosts, not intermediate routers, choose the path a packet takes*. The network is organized into *isolation domains* (ISDs), each governed by a set of core autonomous systems (ASes). Path segments are discovered by a beaconing process: core ASes flood *path-segment construction beacons* (PCBs) that accumulate hop information as they propagate, giving rise to three segment types – *up* segments from a leaf AS to its ISD core, *core* segments between core ASes across ISDs, and *down* segments from a core to a destination leaf. An end host combines an up, an (optional) core, and a down segment into a complete forwarding path and embeds it in the packet header.

Two properties of SCION matter for this thesis. First, **path multiplicity**: a source– destination pair is typically connected by *several* distinct paths that differ in latency, available bandwidth, loss, and the ASes they traverse. This is precisely the raw material an intent-conditioned path-selection agent needs, and the same multiplicity makes multipath transport natural. Second, **path transparency and trust**: because the full AS-level path is visible to the endpoint, an agent can reason about properties beyond raw performance – for instance a *trust* score that reflects how much confidence to place in a path's advertised or measured quality. We exploit this in our single-path selection studies, where the reward balances measured goodput against a trust term. **[TODO:** *the "trust" term used in Chapter 4 is defined purely as* $1 - (w_3 \,\text{loss} + w_4 \,\text{delay})$ *(Eq.* (4.1)*) – a reliability score derived from measured loss and delay, with no AS reputation, certificate, or path-authentication*

content. Either rename it ("reliability", "path quality") throughout the thesis, or state explicitly here and at Eq. (4.1) *that it is a performance proxy and not trust in the SCION control-plane sense. A reader from this group will otherwise read the stronger meaning into it.*] The flip side of endpoint path choice is a coordination problem: whereas a router runs one routing process, every SCION host must decide for itself, ideally without flooding the network with probes. Minimizing that probing overhead is an explicit objective throughout our work.

[**TODO:** *this chapter is missing three pieces of background that Chapter 4 uses without introducing. (i)* Topology generation*: the study's environment is built with BRITE and a Barabási–Albert AS graph model, which is never explained. (ii)* The heuristics used as baselines*: equal-cost multi-path (ECMP) and what a "default" SCION path policy is (shortest AS-path, tie-broken by latency) both need a sentence here, since §4.3.3 compares against them. (iii)* Prior SCION path selection*: how endpoints choose paths today – the path policy language, the path service, and existing selection heuristics – so that the learned selector has a stated status quo to improve on.*]

2.2 Quality of Experience and Transport Mechanisms

2.2.1 Quality of experience for real-time video

To evaluate end-to-end intent control, we target interactive video, whose value to a user is captured by quality-of-experience (QoE) rather than any single network metric. Perceptual quality at a given encoding is commonly summarized by VMAF [10], a learned metric that predicts subjective video quality; interactivity adds extreme sensitivity to end-to-end latency, jitter, and frame loss, since a late frame in a live call is effectively a lost frame. Our multipath reward composes a VMAF-based quality term with penalties for latency, jitter, and loss, making the QoE objective explicit and tunable.

2.2.2 Adaptive bitrate and learned control

The application-side lever in joint control is adaptive bitrate (ABR) control: choosing the video encoder's target rate to match available capacity. Learned ABR controllers such as Pensieve [11] showed that RL can outperform hand-crafted heuristics for buffered video streaming. Our setting differs in two critical ways: it is strictly real-time (lacking a large playback buffer to hide network variance), and the bitrate decision is tightly coupled to a simultaneous *network-side* decision regarding how to allocate traffic across multiple paths.

2.2.3 Multipath QUIC

QUIC [8] is a modern, encrypted, multiplexed transport running over UDP. Multipath QUIC [5] extends it to send a single connection's data across several

network paths at once, in the spirit of Multipath TCP [6] but with user-space deployability. Over a path-aware substrate like SCION, multiple paths are readily available from path discovery, making MPQUIC a natural transport for exploiting SCION's path multiplicity. The open question MPQUIC raises – how to *schedule* bytes across heterogeneous paths – is one of the primary control dimensions learned in this thesis.

2.3 Deep Reinforcement Learning Machinery

2.3.1 Markov decision processes

We model path and rate control as sequential decision-making in a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where at each step the agent observes a state $s \in \mathcal{S}$, takes an action $a \in \mathcal{A}$, receives a scalar reward $r(s, a)$, and transitions to s' with probability $P(s' \mid s, a)$. The agent seeks a policy π that maximizes the expected discounted return $\mathbb{E}\left[\sum_t \gamma^t r_t\right]$ with discount $\gamma \in [0, 1)$. The reward function is where application intent enters: by changing what the agent is rewarded for, we change what "best" means.

2.3.2 Deep Q-networks and refinements

The action-value function $Q^\pi(s, a) = \mathbb{E}[\sum_t \gamma^t r_t \mid s_0 = s, a_0 = a]$ gives the expected return of taking a in s and following π thereafter. A deep Q-network (DQN) [12] approximates the optimal $Q^\star$ with a neural network trained to minimize the temporal-difference error against a slowly updated *target* network, sampling transitions from a replay buffer to decorrelate them. We utilize three standard refinements. *Double DQN* [18] decouples action selection from action evaluation in the bootstrap target, reducing the overestimation bias of vanilla DQN. The *dueling* architecture [19] factors $Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s, a')\right)$ into a state-value stream V and an advantage stream A, which stabilizes learning when many paths have comparably good value. *Prioritized experience replay* (PER) [15] samples high-error transitions more often, correcting the resulting bias with importance-sampling weights.

2.3.3 Soft actor-critic

For continuous control domains – such as determining target bitrates and continuous traffic split ratios – value-based DQN is ill-suited. We adopt soft actor-critic (SAC) [7], an off-policy, maximum-entropy actor-critic method. SAC augments the reward with a policy-entropy term, $\mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot \mid s_t)))]$, which encourages exploration and robustness; it trains twin Q-critics to curb overestimation and automatically tunes the temperature α against a target entropy. SAC's sample

efficiency and stability make it a strong fit for expensive, noisy rollouts in network simulators.

2.3.4 Variable and structured action spaces

A central difficulty in network path control is that the number of candidate paths is dynamic: it varies across source–destination pairs and changes over time. A classical neural network with a fixed output head cannot represent this unbounded decision space. The remedy, used throughout this thesis, is a *scoring* architecture that is *permutation-equivariant* over the path set: a shared network maps each path's feature vector (together with a global context) to a scalar score, and the scores are compared – by arg max for discrete single-path selection, or by a softmax to produce a continuous traffic split. Invalid or absent paths are masked out. Aggregating a set of per-element embeddings in a way that is invariant to their ordering is achieved via the Deep Sets construction [22], which we employ in our continuous critic design. This structural approach decouples the policy from the cardinality of the action set, allowing a single network to generalize across topologies with arbitrary numbers of paths.

2.4 Goal-Conditioned Reinforcement Learning

2.4.1 Conditioning a policy on its objective

A policy that must serve diverse application preferences can be made a function of the objective itself. In goal-conditioned RL, the policy $\pi(a \mid s, g)$ and value function are conditioned on a goal vector g, so that a single neural network represents a whole family of behaviors. Universal value function approximators (UVFAs) [14] formalize this by learning $V(s, g)$ jointly over states and goals, while hindsight experience replay [2] improves sample efficiency by relabeling trajectories with goals achieved *a posteriori*. In this thesis, the "goal" represents application intent, encoded as a vector of reward weights: the agent is explicitly told how much it should prioritize throughput versus latency, jitter, loss, or trust, and is expected to adapt its decisions dynamically.

2.4.2 Where the conditioning signal enters

Where the goal vector enters the network is as consequential as whether it is supplied at all, and the literature on goal-conditioned RL says little about it because its usual policies are not comparison-based. Our policies are. When the action is an arg max over per-alternative scores, a conditioning signal that enters only terms shared by every alternative shifts all scores together and cancels under the comparison, so the policy can consume the goal without ever acting on it. The signal must instead reach the per-alternative terms, where it can interact with

each candidate's own features and change their relative order. Section 4.2.1 makes this precise as Proposition 4.1, and §4.4.2 measures its consequences.

3 Problem Statement & Desired Properties

This chapter formalizes the network decision problems addressed in this thesis. We define the system model, state the operational decision spaces across single-path and multipath regimes, enumerate the essential properties a satisfactory solution must exhibit, and outline our core operating assumptions and trust boundaries.

3.1 Problem Formulation

Consider an endpoint host communicating over a path-aware architecture such as SCION. At any decision epoch t, the host discovers a dynamic set of N_t candidate end-to-end paths $\mathcal{P}_t = \{p_1, \ldots, p_{N_t}\}$. Each path p_i is characterized by partially observable and dynamic performance properties – latency, loss rate, available bandwidth, hop count, and path trust. Simultaneously, the running application expresses its *intent* $g \in \mathcal{G}$, defined as a declarative preference vector prioritizing specific performance metrics (e.g., maximizing throughput, minimizing delay, or optimizing video quality).

Because network utilization and link availability fluctuate over time, the host must dynamically determine how best to utilize the path set $\mathcal{P}_t$ to serve intent g. We formulate this decision across two progressive operational settings:

1. **Intent-Aware Single-Path Selection (Discrete Decision):** For general application flows constrained to a single path, the decision is a discrete selection $p^\star \in \mathcal{P}_t$. The objective is to maximize an intent-weighted reward balancing goodput, latency, and trust while minimizing the active probing overhead required to inspect candidate path states (Chapter 4).

2. **Joint Application–Network Multipath Optimization (Continuous Decision):** For high-bandwidth, latency-sensitive applications (e.g., real-time WebRTC video) operating over Multipath QUIC, the decision space expands. The endpoint must jointly choose a continuous traffic split $\boldsymbol{w} = (w_1, \ldots, w_{N_t})$ across paths (where $\sum w_i = 1$) *and* adapt the application's own sending rate (the video encoder's target bitrate R_{video}) to maximize real-time Quality of Experience (QoE) (Chapter 5).

In both formulations, the technical crux is twofold: (i) the objective is dynamic and dictated by an intent g that varies across flows, and (ii) the action space

is dynamic and unbounded, as candidate path sets $\mathcal{P}_t$ fluctuate across network topologies and over time.

3.2 Desired Properties

A satisfactory solution must exhibit six key properties, which also define the primary metrics evaluated in subsequent chapters (Chapters 4–6):

For the single-path system, each of the six is measured rather than asserted: §4.4.3 for Intent Alignment, §4.4.4 for Zero-Shot Generalization, and §4.4.5 for Dynamic-Action Handling, with §4.4.2 reporting Structural Generalization on a disjoint set of source–destination pairs. One qualification applies throughout: structural generalization is demonstrated across pairs and candidate-set sizes within a single topology, since only one topology was generated, so generalization across topology *scale* remains a design property rather than an evaluated one.

Intent Alignment. The policy must faithfully reflect the specified preference vector g. Decisions under g_{latency} must prioritize low delay, whereas decisions under $g_{\text{throughput}}$ must prioritize goodput. A policy whose action choices remain invariant to changes in g fails this requirement.

Zero-Shot Generalization Across Objectives. A single trained network must serve the entire spectrum of application intents without requiring per-objective model retraining or fine-tuning.

Dynamic-Action Handling. The policy architecture must natively accept dynamic candidate path counts N_t without structural modification or retraining, remaining invariant to the arbitrary ordering of paths in $\mathcal{P}_t$.

Low Overhead. Decision-making must remain computationally lightweight. In single-path selection, active probing cost must be minimized; in multipath real-time control, per-frame inference latency must fit comfortably within frame processing budgets ($\approx 33\,\text{ms}$).

Structural & Topological Generalization. Performance must hold off-distribution across unseen topologies, dynamic path counts, and shifting traffic workloads.

Robustness to Network Volatility. The policy must sustain high performance under non-stationary conditions – path churn, sudden link failures, and transient congestion bursts – where static heuristics fail.

3.3 Assumptions & Scope

We explicitly state the operating assumptions underlying our design:

1. **Path-Aware Substrate.** The underlying network exposes multiple distinct end-to-end paths to the endpoint and honors source routing decisions, as provided by SCION. We do not assume control over internal intra-AS routing decisions.

2. **Observable Path Feedback.** Per-path metrics (latency, loss, bandwidth) are measurable via active probing (in single-path selection) or derived continuously from transport feedback (e.g., QUIC ACK frames and RTT samples).

3. **Explicit Intent Declaration.** Application intent is declared as an explicit preference vector (e.g., reward profile weights or target QoE parameterizations). We do not attempt to infer latent intent from raw packet inspection.

4. **Episodic Stationarity.** Network state is assumed roughly stationary within a brief decision epoch, though conditions may drift or shift abruptly between epochs.

4 Motivating Study: Intent-Conditioned Single-Path Selection

This chapter investigates the first and most basic control decision a SCION endpoint faces: given a running application's intent and a changing network, *which* of the candidate paths to a destination should carry a flow? We answer it in five steps. Section 4.1 builds the selection framework – the simulation environment, the decision problem, and a permutation-equivariant scoring architecture that copes with a variable path set. Section 4.2 adds the chapter's central contribution: we show that *where* an intent signal enters a comparison-based policy decides whether it can change the chosen path at all, and condition the per-path features accordingly. Section 4.3 describes the implementation, the baselines, and the per-context oracle against which everything is measured. Section 4.4 evaluates the conditioned selector on held-out hours, on intents it never trained on, and on candidate sets whose size it never saw. Finally, §4.5 identifies the single-path performance ceiling that motivates the multipath system of Chapter 5.

4.1 Selection Framework: Environment, Decision, and Scoring

We start from the DQN path selector of Bourak et al. [3] and identify three limitations we set out to remove (§4.1.1). We then describe the improved simulation environment (§4.1.2), the state, action, and reward that define the decision problem (§4.1.3), and the permutation-equivariant path-scoring architecture that handles a variable path set (§4.1.4).

4.1.1 Starting Point and Its Limitations

Bourak et al. [3] frame SCION path selection as a reinforcement-learning problem and train a DQN to pick a path from per-path measurements of latency, loss, and bandwidth, rewarding a weighted combination of goodput and trust while charging for probing. They report oracle-level selection quality at a fraction of the probing cost of measuring every path. This validates the RL framing and gives us a reward structure to build on, but three design choices limit it.

Fixed action space. A vanilla DQN has one output per action, so the number of paths must be fixed and known. Real source–destination pairs have different,

time-varying path counts; padding to a maximum wastes capacity and still caps the topology size.

Single baked-in objective. The reward weights are constants chosen before training. The resulting policy embodies *one* trade-off between goodput and trust; a different application intent demands a different, separately trained model.

Environment fidelity. Capturing when path selection actually matters requires paths that genuinely share bottlenecks and conditions that vary over realistic timescales, so that a good choice at one hour is a poor choice at another.

Our design removes all three: a scoring architecture for the first, per-path intent conditioning for the second, and an improved environment for the third. A strawman for the second limitation – appending the reward weights to the state of the existing DQN – is instructive precisely because of *where* it fails, as we show in §4.2.

The third limitation deserves a number rather than an assertion, since it bounds the size of every effect this chapter reports. The quantity that settles it is the spread of achievable goodput across a pair's candidate paths *within a single decision context* – exactly what a selector has to exploit. We measure it in §4.1.2 and find it wide and time-varying, so path choice is a first-order effect in this environment.

4.1.2 Improved SCION Simulation Environment

Training and evaluating a path selector requires an environment in which the "right" path changes with conditions. We construct one in several stages.

Topology. AS-level topologies are generated with BRITE and converted into SCION structures – ISDs, core and non-core ASes, and inter/intra-ISD links – so that the graph has realistic degree and locality structure rather than being hand-drawn. We describe the conversion concretely, because the capacity heterogeneity the agent exploits comes from the converter rather than from BRITE, and a reader who assumes BRITE-generated bandwidths will misread the results. BRITE runs its AS-level Barabási–Albert model on 90 nodes with the bandwidth distribution pinned to a constant, so *every one of its 382 links has exactly 10 Gbit/s of capacity*. The converter assigns ISDs by k-means on node coordinates and core ASes by degree rank, then adds 146 links of its own with hard-coded capacities (Table 4.1); these additions are the sole source of capacity variation in the graph. Link latency is likewise not BRITE's delay column, which is on an abstract scale, but $\max(0.5,\ 0.02\,d)$ milliseconds in the node coordinate plane, giving a median inter-AS latency of 8.5 ms and a maximum of 23.0 ms.

Control plane. A beaconing simulation propagates path-segment construction beacons top-down within each ISD and across the core mesh between ISDs,

Table 4.1: Link capacities in the evaluated topology, by origin. BRITE contributes a uniform 10 Gbit/s fabric; every capacity difference the selector can exploit is introduced by the SCION converter.

Origin	Link role	Capacity	Count
BRITE	peer, parent–child, core	10 Gbit/s	382
SCION converter	multi-parent	1 Gbit/s	35
	cross-connect	2 Gbit/s	35
	additional peering	$\mathcal{U}(5, 10)$ Gbit/s	75
	inter-ISD core	50 Gbit/s	1
Total			528

yielding the up/core/down segments that compose into end-to-end paths. Peer links are excluded. The result is, for each source–destination pair, a realistic set of candidate paths of varying length and quality – the agent's action set.

Traffic and link dynamics. We drive the network with 28 days of hourly demand: foreground flows for every routable pair plus randomized background traffic, modulated by diurnal and weekly factors. Crucially, load is aggregated *per link*, so paths that share a bottleneck link contend for its capacity. This is what makes selection non-trivial: the utilization and effective bandwidth of a path depend on what every other flow is doing at that hour, and the best path at 3 a.m. is often congested at 8 p.m.

Link and path model. A link's utilization is its aggregate offered load over its capacity, capped at 1.5; its latency is a base propagation delay scaled by a queueing multiplier that grows with utilization and saturates at 2×; and its loss rate is a piecewise function of utilization alone, zero below 0.8 and capped at 0.2. A path's latency is the sum over its links, its loss the complement of the product of per-link survival probabilities, its utilization the maximum over its links, and its *available bandwidth* the minimum residual capacity $\max(0,\ C_e - \text{load}_e)$ across its links.

Two consequences of this model bound how the chapter's results should be read, and we state them here rather than leave them to be discovered. First, the model is *open-loop*: the agent's own selection never adds load to the links it traverses, so what the reward calls goodput is the spare capacity at the bottleneck rather than a rate the flow would actually achieve, and there is no per-flow fair share or congestion feedback. Second, utilization is capped at 1.5 rather than 1.0, so links can be oversubscribed; this is what produces the > 1.0 abscissa values in Fig. 4.6. Both are defensible choices for a study of *selection* – which path is best is well defined without modeling the flow's own back-pressure – but neither would survive into a closed-loop transport study, and Chapter 5 accordingly uses a different data plane.

Does path choice matter here? The environment is only interesting if the candidate paths of a pair differ materially at a given hour, so we measure that spread directly over all 10752 held-out decision contexts (§4.4.1). Within a single context, the best candidate carries **3.04**× the median candidate's goodput (p10 1.57×, p90 18.26×) and 7983 Mbit/s more than the worst; the coefficient of variation of goodput across candidates has median 1.43, and the worst candidate's latency exceeds the best's by 29.3 ms at the median, a factor of 3.5. The choice is also genuinely time-varying: the *identity* of the highest-goodput path changes between consecutive hours in 8.6% of cases, so a policy cannot memorize one answer per pair. At the network level, p90 link utilization is 0.76 with 7.8% of links oversubscribed, and only 0.09% of contexts have no usable bandwidth on any candidate. Path choice is therefore a first-order effect in this environment, which is the premise §4.1.1 needs.

Probing model. Fresh metrics are not free. The environment charges a latency probe 10 ms plus 0.5 ms per hop, and a full bandwidth probe 100 ms plus 20 ms per hop; it credits the agent with having probed only the path(s) it actually inspects. Because both costs grow with hop count, probing is cheaper on short paths – a fact §4.4.6 returns to. The reward charges the *mean* cost per probe issued in a step, normalized by a 500 ms reference. This turns "measure less" into a real objective rather than an afterthought, and lets us compare the learned selector's probing footprint against baselines that probe exhaustively.

Charging the *mean* rather than the total cost is consequential and deserves justification, because it means a method issuing thirty cheap probes is penalized less by the reward than one issuing a single expensive probe. This is deliberate: the per-probe charge measures cost per unit of information gained, which keeps the reward a statement about *selection quality* and prevents the probing term from dominating the goodput and trust terms it is meant to modulate. The consequence is that the reward understates the probing gap – under the Balanced intent the learned selector is charged 135 ms per probe against exhaustive widest-path selection's 180 ms, a near-tie, while their *total* measurement cost differs by a factor of 40. We therefore report total probing cost separately in §4.4.6 rather than relying on the reward to express it.

4.1.3 Decision Problem: State, Action, Reward

State. The agent observes a *global* context and a *per-path* feature matrix. The global context summarizes time and aggregate conditions – normalized day-of-week and hour-of-day, mean path utilization, mean trust, and the fraction of paths whose utilization exceeds 0.7 – plus a compact encoding of the source and destination. The three aggregate terms are computed over the current flow's own candidate path set, not over the whole network. Each candidate path contributes a seven-dimensional feature row: latency normalized by 100 ms, loss rate, hop

count over 20, the path's bandwidth relative to the best candidate at that hour, utilization, a static-bandwidth term, and a trust score. Because the number of rows N varies, the state is naturally a (global, path-matrix) pair rather than a flat vector.

Action. The action is the choice of one path, i.e. an index into the current candidate set. Because N varies, the action is realized as an arg max over per-path scores (§4.1.4) with invalid entries masked, rather than as a fixed categorical head.

Reward. We adopt and generalize the goodput–trust reward of Bourak et al. [3]. For the chosen path, let goodput $\in [0, 1]$ be the achieved bandwidth normalized by the *best available bandwidth among that pair's candidate paths at that hour*, and define a trust term from loss and delay,

$$\text{trust} = \max\big(0,\ 1 - (w_3\,\text{loss} + w_4\,\text{delay})\big), \tag{4.1}$$

with delay the latency capped at 100 ms and normalized. The base reward trades goodput against trust and is shifted to $[-1, 1]$, and a probing penalty is subtracted:

$$r = \underbrace{2\big(w_1\,\text{goodput} + w_2\,\text{trust}\big) - 1}_{\text{base reward}} - w_{\text{probe}} \cdot \text{probe\_penalty}, \tag{4.2}$$

clamped to $[-1, 1]$, where probe_penalty is the probing cost normalized to $[0, 1]$. We take $w_1 + w_2 = 1$ throughout, which is what places the base reward in $[-1, 1]$ before clamping. The five weights $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ are exactly the *application intent*: default balanced values are $(0.7, 0.3, 0.5, 0.5, 0.05)$, but a throughput-hungry intent pushes w_1 toward 1, a loss-averse intent raises w_2 and w_3, a latency-sensitive intent raises w_4, and a probe-frugal intent raises w_{probe}. The design goal of §4.2 is a *single* policy that behaves correctly for *any* setting of $\mathbf{w}$.

Two properties of this reward shape every result in the chapter, and we state them explicitly because both are easy to misread. First, goodput is normalized against the *best candidate path of that pair at that hour*, so it is a relative score rather than an absolute rate: a policy that always picks the best available path scores near 1 whether that path carries 9 Gbit/s or 2 Gbit/s. The reward therefore measures selection quality and is blind by construction to how much capacity exists to select from – which is why, in §4.5, the learned selector's reward stays flat across the congestion range while its goodput falls by a quarter. Second, latency enters the objective only *inside* the trust term, through w_4; there is no standalone latency weight. A latency-sensitive intent is expressed by raising w_2 and w_4 together, which is how the *Low-Latency* profile of Table A.1 is defined.

4.1.4 Handling a Variable Path Set: The Scoring Architecture

To make the policy independent of the path count, we score each path with a *shared* network and select by comparison, following the permutation-equivariant

design of §2.3.4. The global context is broadcast and concatenated to every path's feature row, and a shared multilayer perceptron maps each (global, path$_i$) pair to a scalar Q-value. Selection is $\arg\max_i Q_i$; absent or invalid paths are masked to -10^9 so they can never be chosen and do not distort training targets.

We use a *dueling* scoring head (§2.3): a value stream reads the global context alone to produce $V(s)$, an advantage stream reads the per-path features to produce $A(s, i)$, and the two combine as $Q_i = V(s) + \big(A(s, i) - \text{mean}_{j \in \text{valid}} A(s, j)\big)$ with the mean taken only over valid paths. Because many paths are often comparably good, separating "how good is this situation" from "which path is marginally better" stabilizes learning. Double DQN targets and prioritized replay (§2.3) further stabilize and accelerate training. The scoring design also delivers the dynamic-action property directly: the same weights serve a pair with three paths and a pair with thirty, and are invariant to how the paths happen to be ordered.

4.2 Where Intent Conditioning Must Enter

The central contribution of this chapter is making one scoring policy serve every intent $\mathbf{w}$. The finding is that this is not a matter of feeding the policy its intent: *where* the intent enters decides whether it can influence the choice at all.

4.2.1 Where conditioning cancels

The obvious approach is to append $\mathbf{w}$ to the state. Whether that works depends entirely on *which* part of the state it is appended to, and the reason is a one-line argument about the comparison that defines the action.

Proposition 4.1 (Common-mode cancellation)**.** *Let a scoring policy assign path i the score $Q_i(s, \mathbf{w}) = f(s, f_i) + c(s, \mathbf{w})$, where f_i is path i's feature row and the conditioning enters only through the term c, which does not depend on i. Then* $\arg\max_i Q_i(s, \mathbf{w})$ *is independent of* $\mathbf{w}$*: the policy consumes the intent but its action is invariant to it.*

The proof is immediate — $c(s, \mathbf{w})$ is a constant offset across the compared alternatives, so it cancels in every pairwise comparison $Q_i - Q_j$ and cannot change which index attains the maximum. The point is that this is not a statement about optimization difficulty or sample efficiency: no amount of training can overcome it, because the intent-dependent term is not in the image of the decision rule at all.

The dueling scoring head of §4.1.4 instantiates exactly this hypothesis when the intent is routed into the value stream. There $Q_i = V(s, \mathbf{w}) + \big(A(s, i) - \text{mean}_j A(s, j)\big)$, so V plays the role of c: conditioning the value stream is provably inert. We verify this directly at the network level – holding path features fixed and varying only $\mathbf{w}$ leaves every pairwise Q-difference constant to within 10^{-5} – and §4.4.2 confirms the behavioral consequence over five training seeds.

One qualification matters, because it is the difference between "cannot re-rank" and "re-ranks only through a channel it does not control". Proposition 4.1 is a

statement about the *architecture*, and in the deployed system its premise is mildly violated: the seventh per-path feature is the trust score, which is itself computed from w_3 and w_4. Intent therefore does leak into f_i through one feature, and this is precisely why the value-stream variant of §4.4.2 attains a small but nonzero behavioral divergence rather than exactly zero. The measured residual is what the leak predicts, which sharpens the proposition rather than weakening it.

4.2.2 Conditioning the per-path comparison

The fix follows directly from Proposition 4.1: the intent must reach the term that distinguishes one candidate from another. The scoring architecture of §4.1.4 already broadcasts the global context to every path row before the advantage stream, so it suffices to make the intent part of that context. Writing $\tilde{s} = [\, s \,\|\, \mathbf{w} \,]$ for the widened global vector, with $[\,\cdot \,\|\, \cdot\,]$ denoting concatenation, the scores become

$$Q_i = V(\tilde{s}) + \Big(A\big([\, \tilde{s} \,\|\, f_i \,]\big) - \mathrm{mean}_{j \in \mathrm{valid}}\, A\big([\, \tilde{s} \,\|\, f_j \,]\big) \Big). \tag{4.3}$$

The advantage stream now evaluates $\mathbf{w}$ jointly with each path's own feature row, and because the shared scorer is nonlinear, the same $\mathbf{w}$ shifts two candidates by different amounts, so the ordering can change. This is not of the form Proposition 4.1 assumes and the cancellation argument does not apply. The intent also reaches the value stream, which is why we call the agent *Two-Stream-Concat*; by the proposition that half is inert for the choice, and it is the advantage stream that does the work.

We adopt this mechanism because it is the least the proposition permits. It introduces no conditioning sub-network, no new hyperparameters, and no new architectural component: it widens the existing global context by the five intent weights and lets the existing broadcast do the rest, for $5 \times 128 \times 2 = 1280$ additional parameters – one input weight per hidden unit of each stream. It also inherits the architecture's independence of the path count unchanged, since the intent is broadcast rather than replicated per candidate. Section 4.4.2 shows this captures 93% of the re-ranking a per-context oracle achieves, which leaves little room for a more elaborate mechanism to improve on. The finding this chapter reports is about *where* the intent enters, not about the machinery that carries it once it is in the right place.

4.2.3 Training across intents

To learn a policy that spans the intent space we train across six *reward profiles* – named weight vectors spanning a throughput extreme, a loss-averse extreme, a latency-averse extreme, a balanced midpoint, and two probe-frugality settings (Table A.1). Each training episode is assigned a profile, sets the environment's reward weights to it, and feeds the same weights to the scorer, so the agent simultaneously experiences the reward for an intent and the encoding of that

intent. Profiles are scheduled in a stratified fashion – each profile repeated to fill the episode budget, then shuffled – so every intent receives an equal number of episodes. The intent is presented to the network in a rescaled encoding rather than as raw weights, which keeps the conditioning inputs on a comparable scale to the path features.

Training on a finite set of profiles is a means, not the goal. What we want is a policy that maps the *intent space* to behavior, so that an unseen $\mathbf{w}$ presented at inference – in particular one interpolating between two trained profiles – yields correspondingly interpolated behavior without retraining. This is the zero-shot generalization property required by §3.2, and §4.4.4 tests it directly.

4.2.4 Relation to Mortise

This is the path-selection instance of Mortise's principle [16]. Where Mortise maps an application's revealed QoS preference to congestion-control *parameters*, our scorer makes an explicit intent vector an input to the per-path comparison that determines which path is chosen. In both, intent is a first-class input that reshapes a network mechanism, not a constant compiled into it. Chapter 5 pushes the same principle one step further, letting intent shape not only the network's choice but the application's own output.

4.3 Implementation

This section describes how the design is realized: the agent family and their architectures (§4.3.1), the end-to-end evaluation pipeline that turns a topology into trained models and comparable results (§4.3.2), and the baselines and probing accounting against which the learned selectors are measured (§4.3.3). The implementation is in Python (PyTorch for the agents) and is organized so that each pipeline stage passes data to the next through timestamped run directories.

4.3.1 Agent Family

The codebase implements a family of agents of increasing capability, from which the ablation ladder of §4.4.2 is drawn.

Flat DQN (simple / enhanced). A conventional fixed-action DQN over the 5-dimensional global state. The *simple* variant is a small multilayer perceptron with ε-greedy exploration and a periodic target network; the *enhanced* variant adds Double DQN [18], a dueling head [19], prioritized experience replay [15], soft target updates, and action masking. These agents cannot handle a variable path set and stand in for the original design [3].

Path-scoring DQN (simple / enhanced). The permutation-equivariant design of §4.1.4: a shared per-path scorer producing one Q-value per path, with

variable N handled by padding to the batch maximum and masking. The enhanced variant uses the dueling scoring head, Double DQN, and prioritized replay. This family delivers the dynamic-action property.

Conditional path-scoring DQN (*Two-Stream-Concat*). The recommended agent, adding the conditioning of §4.2.2: the five-dimensional encoded intent widens the global context, which the scoring head already broadcasts to both streams, so the advantage stream evaluates the intent jointly with each candidate's features. Two intent encodings are supported and recorded in the checkpoint – the raw weights, or a policy-friendly rescaling used in all reported runs.

Value-Concat. The control that isolates Proposition 4.1. It is identical to the above except that the intent is routed to the value stream alone, leaving the advantage stream on the unwidened context and the raw path features; by the proposition it cannot re-rank.

All five rungs were trained on the same environment, the same 28-day demand trace, and the same optimizer settings, so the ladder isolates one architectural change at a time: flat to scoring (per-path features and a variable action set), scoring to Value-Concat (intent present but path-independent), and Value-Concat to Two-Stream-Concat (intent reaching the per-path comparison).

Training details. All agents use two hidden layers of width 128, learning rate 10^{-3}, discount $\gamma = 0.95$, batch size 32, a replay buffer of 10^4 transitions, and soft (Polyak) target updates with $\tau = 0.05$. Prioritized replay uses priority exponent $\alpha = 0.6$ with the importance-sampling exponent β annealed from 0.4 to 1.0. Exploration is ε-greedy, decayed multiplicatively per episode. Invalid actions are masked to a large negative value both when acting and when forming bootstrap targets, so masked paths never contribute to the loss. Episodes are 24 steps (one simulated day); the flat agent trains for 533 episodes and the conditional agents for 666, with one gradient step every four environment steps. The conditional agents are trained across reward profiles on a stratified schedule (§4.2), so that within a single training run the network sees every intent an equal number of times.

This budget is short, and we state its consequences rather than leave them implicit. At one gradient step per four environment steps, 533 and 666 episodes amount to roughly 3200 and 4000 optimizer updates in total, and ε decays only to 0.069 (flat) and 0.036 (conditional), never reaching the stated 0.01 floor. The natural worry is that the weakest rung of the ladder is undertrained rather than architecturally limited, which would inflate the gap the ablation attributes to the architecture. The training curves (Appendix A.4) settle it: the flat DQN plateaus at a mean episode reward of +0.049 with a training loss that *rises* over the run, while the unconditioned scoring agent, given the identical 533 episodes on the identical data with the identical optimizer, reaches +0.809 – sixteen times the final reward. The flat agent's deficit is a property of its architecture, a fixed 30-way

action head over a five-dimensional global state with no per-path features, not of its budget. The conditional agents converge to +0.814 (Two-Stream-Concat) and +0.816 (Value-Concat) over 666 episodes. Every agent in the ladder was retrained under five random seeds, and §4.4.2 reports the resulting spread.

4.3.2 Evaluation Pipeline

The system is driven by a six-stage pipeline, each stage writing artifacts consumed by the next:

1. **Topology generation** – BRITE produces an AS graph, converted into SCION roles, ISDs, and links.
2. **Beaconing** – PCB propagation discovers up/core/down segments and composes candidate paths per source–destination pair.
3. **Traffic simulation** – 28 days of hourly demand with per-link aggregation, producing the time-varying link states the environment replays.
4. **Training** – all agent variants are trained on the resulting environment; checkpoints and per-episode statistics are saved.
5. **Evaluation** – every method (learned and baseline) is run over held-out pairs, hours, and intents, logging reward, achieved latency and bandwidth, probing overhead, and selection time.
6. **Figures** – results are rendered in a consistent style for the report.

Running the whole pipeline is a single command; intermediate artifacts (topology, path store, link states, checkpoints, results tables) live under a timestamped run directory so that any stage can be re-run or inspected in isolation. Key knobs – topology size, number of training episodes, the cap on training pairs, and learning hyperparameters – are exposed as configuration so that experiments can be scaled up or down.

4.3.3 Baselines and Probing Accounting

Baselines. Six heuristic selectors provide the comparison points. *Shortest-path* takes the fewest AS hops and *lowest-latency* the smallest measured delay. *Widest-path* takes the largest measured available bandwidth, and is the only baseline requiring a full bandwidth probe on every candidate. *ECMP* restricts to the minimum-hop set and pins the flow to one member by hashing its five-tuple, and the *SCION default* takes the minimum hop count with ties broken by latency. Finally *random* fixes one pseudo-randomly chosen path per source–destination pair and keeps it for the whole evaluation, which makes it a static-misconfiguration

reference rather than a per-decision coin flip. Together they span the spectrum from naive to strong single-metric heuristics (widest for throughput, lowest-latency for delay), so that the learned selectors must beat not just a weak baseline but the *right* heuristic for each intent.

Oracle. None of these heuristics is an upper bound. Widest-path maximizes bandwidth, which is the correct target only under the Throughput intent; under Low-Latency it is actively wrong. We therefore also evaluate a per-context *oracle*: for each decision it enumerates the candidate set and selects the path maximizing the *true reward under the intent being scored*. It is charged one full probe, so its probing cost is comparable to the learned selector's, and its goodput is normalized identically – by that pair's best candidate at that hour. The oracle is not a deployable policy, since it requires perfect knowledge of every candidate's realized metrics before choosing; it is the per-intent upper bound that turns Table 4.2 into a normalized optimality gap and lets us say how much of the available quality each method captures.

Probing accounting. A fair comparison must charge each method for the measurements it needs. The environment therefore records, per decision, which paths a method probed and at what cost – latency probes being cheap and full bandwidth probes expensive (§4.1.2). Heuristics that require a metric on every path (e.g. widest-path) incur the cost of probing all paths, whereas the learned selector is charged only for the paths it actually inspects. This makes the probing-overhead reduction a directly measurable quantity rather than an assumed benefit, and is the mechanism by which we test the high-quality-at-low-probing claim of Bourak et al. [3].

Concretely, and since this drives the whole of Fig. 4.5: widest-path is charged a full bandwidth probe on all 30 candidates, 5388 ms per selection; every other heuristic a latency probe on all 30, 360 ms; and the learned selector a full bandwidth probe on the single path it selects, 132 – 139 ms depending on that path's hop count.

One label needs correcting. Our ECMP baseline is evaluated with a constant flow identifier, so it hashes every flow of a pair onto the same member of the minimum-hop set and exhibits none of the load spreading its name implies. It is *hash-pinned shortest path* here, and we call it that where the distinction matters; we retain the ECMP label in tables for comparability with the literature.

4.4 Experimental Evaluation and Intent Alignment

This section evaluates the selectors on held-out hours. We first fix the experimental setup and metrics (§4.4.1). We then answer five questions the design raises, in turn: where must intent enter for conditioning to have any effect (§4.4.2)? Does

the conditioned policy re-rank paths in the *right* direction as the intent changes (§4.4.3)? Does it generalize to intents it never trained on (§4.4.4)? Does the scoring architecture in fact deliver the dynamic-action property it was chosen for (§4.4.5)? And does it retain the high-quality-at-low-probing property of the selector it builds on (§4.4.6)? Section 4.5 then turns to the limit that motivates the next chapter.

4.4.1 Experimental Setup

Environment. Experiments use one BRITE-generated SCION topology of 90 ASes and 528 inter-AS links, partitioned into 2 ISDs with 6 core ASes. Beaconing (§4.3.2) discovers 1004 core segments and 1127 up/down segments, which compose into candidate paths for 8010 routable source–destination pairs, each with up to 30 paths. The network is driven by the 28-day hourly demand model with per-link aggregation (§4.1.2).

Train/test split. The main tables use a *temporal* split: training uses the first 14 days and every number is measured on the last 14 days (336 held-out hours), a window the agents never trained on. Unless noted, each cell aggregates 32 pairs across those 336 hours, i.e. 10752 independent decision contexts. We are explicit that this is a temporal split rather than a pair split, because the 32 evaluation pairs are drawn from the same pool the agents train on. Training episodes sample a pair uniformly from all 8010 routable pairs and visit 644 distinct ones over the run, so only 2 of the 32 evaluation pairs were ever seen; the weakness of this set is not contamination but homogeneity, since all 32 share the same source AS.

Structurally held-out pairs. To test generalization across the topology rather than across time, we additionally construct an exactly disjoint evaluation set: replaying the training RNG identifies the 644 visited pairs, and we sample 32 pairs from the remaining 7366, chosen to span 32 distinct source ASes. No agent has seen any of them. Table 4.4 reports the ablation on this set.

Intents. The conditional agent is trained across the six reward profiles of §4.2 and evaluated on the four *selection-relevant* ones: throughput-maximizing (*Throughput*), latency-averse (*Low-Latency*), loss-averse (*Low-Loss*), and a balanced midpoint (*Balanced*); their exact weight vectors are given in Table A.1. The two probe-frugality profiles are excluded from the conditioning studies because a single-path selector issues one probe per decision regardless of w_{probe}, so the weight has little leverage over the choice; probing cost is instead the subject of §4.4.6.

This exclusion needs a measured justification rather than an assertion, because probe cost is charged *per hop* (§4.1.2) and w_{probe} is therefore not strictly path-independent: a probe-frugal intent does have some incentive to prefer short paths. The incentive is real but negligible. Evaluating all six profiles as intents, the mean chosen hop count of the *oracle* – which bounds what any policy could do – moves

from 1.899 under `probe_minimal` ($w_{\text{probe}} = 0.001$) to 1.881 under `probe_averse` ($w_{\text{probe}} = 0.35$), a shift of 0.018 hops across a 350-fold change in the weight. For scale, the same oracle moves 0.333 hops between Throughput and Low-Latency. Probe frugality thus commands roughly 5% of the leverage of a genuine intent axis, which is too little to discriminate between paths, and the two profiles are near-duplicates of Balanced by construction. They remain in the training schedule, where they cost nothing and broaden the intent space the policy is exposed to, and are excluded from the conditioning studies.

Metrics. For each method we report the intent-weighted reward of Eq. (4.2) and the achieved per-flow goodput and latency. Intent alignment is assessed two ways: by reward under each intent, and by *behavioral divergence* – the degree to which the chosen path changes as the intent vector changes, holding the decision context fixed. Concretely, writing d for the number of distinct paths a method selects across the $K = 4$ intents in a given context and N for that context's candidate count, the divergence is

$$\text{div} = \frac{d - 1}{\min(K, N) - 1} \in [0, 1], \tag{4.4}$$

averaged over contexts, so a method that ignores intent scores 0 and one that picks a different path for every intent scores 1. We also report the *choice entropy*, the Shannon entropy in bits of the distribution of choices across the four intents, which credits a method for spreading its choices evenly rather than merely for making more than one. Probing cost is measured by the per-selection probe accounting of §4.3.3.

4.4.2 Where Must Intent Enter? The Conditioning Ablation

Proposition 4.1 predicts that conditioning routed only through path-independent terms cannot change the selected path. We test that prediction, and separate it from the effect of the scoring architecture, by walking up the four-rung agent ladder of §4.3.1: a Flat DQN with a single baked-in objective; an *unconditioned* path-scoring DQN, which sees no intent at all and isolates what the architecture alone buys; a Value-Concat agent that receives the intent in its value stream only; and Two-Stream-Concat, which routes it to the per-path comparison. Table 4.2 reports the mean reward each attains under each intent, together with its adaptivity, choice entropy, and distance from the per-context oracle.

Per-path scoring buys the goodput; conditioning buys the re-ranking. The two contributions separate cleanly, and the unconditioned scoring agent is what separates them. It reaches 8.21 Gbit/s of goodput against the flat agent's 5.62 Gbit/s, *with no intent input whatsoever* – so the large absolute gap between the flat and conditional agents belongs to the per-path architecture, not to conditioning. Over

Table 4.2: Intent-conditioning ablation. Mean intent-weighted reward under each of the four intents; *Adapt.* is the mean behavioral divergence of Eq. (4.4) (0 = intent-invariant path choice, 1 = a different path per intent), *Ent.* the mean choice entropy in bits, and *Params* the Q-network's parameter count in thousands. The oracle is the per-context reward arg max under the intent being scored, so each column reads as a normalized optimality gap. Each reward cell aggregates 10752 decisions over 336 held-out hours from the single reference run that the rest of the chapter is also anchored on; Table 4.3 gives every method's spread over five independent training seeds.

	Mean reward by intent				Intent sensitivity		
Method	Thpt.	Low-Lat.	Low-Loss	Bal.	Adapt.	Ent.	Params
Flat DQN	0.333	0.701	0.868	0.561	0.011	0.029	85.3 k
Scoring DQN (uncond.)	0.979	0.679	0.937	0.898	0.003	0.009	36.2 k
Value-Concat	0.979	0.708	0.937	0.894	0.050	0.130	36.9 k
Two-Stream-Concat	**0.986**	**0.731**	**0.940**	**0.897**	**0.149**	**0.375**	37.5 k
Oracle	*0.986*	*0.733*	*0.941*	*0.898*	*0.161*	*0.419*	–

five seeds the effect is 8208 Mbit/s [8196, 8220] against 5808 Mbit/s [5711, 5906], an improvement of 41% with the intervals far apart. What conditioning adds is narrower and precisely targeted. On Throughput, Low-Loss, and Balanced the unconditioned scorer is already statistically indistinguishable from the conditioned one, because those three objectives are all served by the same high-bandwidth paths. On Low-Latency – the one evaluated intent that must steer *away* from the bandwidth-maximizing default – it falls to 0.679 while Two-Stream-Concat reaches 0.731, cutting mean chosen latency from 17.0 ms to 12.7 ms at a cost of roughly 960 Mbit/s of goodput. Its adaptivity of 0.003 confirms it is exactly the intent-blind agent it should be.

The flat architecture is the wrong shape. The Flat DQN collapses onto a single trade-off: it scores 0.868 on Low-Loss but only 0.333 on Throughput and 0.561 on Balanced, and its adaptivity of 0.011 confirms it selects essentially the same path whatever intent it is asked to serve. Its reward variance under Throughput is 0.74, against 0.03 for the unconditioned scorer and 0.003 for the conditioned one – it is not merely worse on average but wildly inconsistent. As §4.3.1 establishes, this is architectural rather than a training artifact.

Where the intent enters decides whether it acts. Both conditional agents receive the same intent vector and both recover strong reward on every intent. The decisive difference is how much they *act* on it: Two-Stream-Concat attains an adaptivity of 0.149 and a choice entropy of 0.375 bits against Value-Concat's 0.050 and 0.130 – a

factor of three in re-ranking. This is the predicted signature of Proposition 4.1: an intent entering only the value stream shifts every path's score by the same amount and cancels under the arg max, so Value-Concat can influence its own choice only through the indirect effect of training and through the single trust feature that leaks w_3, w_4 into f_i. The behavioral consequence shows up as reward exactly where re-ranking is required – on Low-Latency, 0.731 against 0.708.

How much re-ranking is available? Adaptivity lives on $[0, 1]$, so 0.149 looks small in isolation. The oracle sets the scale: a policy with perfect knowledge, re-optimizing per context under each intent, attains an adaptivity of only 0.161 in this environment, because most contexts have no better answer to give a different intent. Two-Stream-Concat therefore captures **93%** of the achievable re-ranking, and Value-Concat 31%.

Distance from optimal. Reading Table 4.2 against its oracle row, Two-Stream-Concat's shortfall is 0.0002 on Throughput, 0.0018 on Low-Latency, 0.0012 on Low-Loss, and 0.0005 on Balanced – at most **0.25%** of the oracle's own reward on any evaluated intent. The conditioned selector is thus near-optimal in the strict sense of being close to a per-context arg max that knows every candidate's realized metrics in advance, while itself inspecting one path.

Reproducibility across seeds. Several of the comparisons above are small reward differences, so we retrained every agent in the ladder under five random seeds and re-evaluated each on the identical 10752 contexts. The environment's pair, hour, and profile streams are held fixed, so every seed sees the same stream of training contexts and only the learning process varies. Table 4.3 reports means with 95% confidence intervals on the two intents that discriminate, plus adaptivity.

Both structural claims survive with room to spare. The architectural one: on Throughput the flat agent's interval [0.3518, 0.3942] is nowhere near the scoring agent's [0.9754, 0.9813]. The conditioning one: on Low-Latency, Value-Concat's [0.6996, 0.7119] and Two-Stream-Concat's [0.7279, 0.7312] are disjoint, as are their adaptivity intervals [0.0309, 0.0541] and [0.1293, 0.1503]. So is the comparison that isolates what conditioning buys – Two-Stream-Concat's [0.7279, 0.7312] against the unconditioned scorer's [0.6755, 0.6841] on the one intent that conflicts with the bandwidth-maximizing default.

One ordering in Table 4.2 does *not* reproduce, and we flag it rather than leave it to be discovered: on Throughput, the reference run has Value-Concat marginally ahead of the unconditioned scorer (0.9788 against 0.9787), while over five seeds the scorer leads (0.9784 against 0.9760) with overlapping intervals. The two are indistinguishable on that intent, which is what the surrounding text claims; neither the single-run nor the five-seed ordering should be read as a result.

A paired Wilcoxon signed-rank test over the 10752 shared contexts agrees with the seed analysis on every comparison above ($p < 10^{-12}$ throughout). We note

Table 4.3: Seed variance over five training runs per method, evaluated on the same 10752 held-out contexts. Cells are mean [95% confidence interval]. Both of the chapter's structural comparisons reproduce with disjoint intervals: flat against per-path scoring on Throughput, and value-stream against per-path conditioning on Low-Latency and on adaptivity.

Method	Throughput reward	Low-Latency reward	Adaptivity
Flat DQN	0.3730 [0.3518, 0.3942]	0.7026 [0.6918, 0.7134]	0.0134 [0.0032, 0.0235]
Scoring DQN (uncond.)	0.9784 [0.9754, 0.9813]	0.6798 [0.6755, 0.6841]	0.0056 [0.0033, 0.0078]
Value-Concat	0.9760 [0.9722, 0.9798]	0.7058 [0.6996, 0.7119]	0.0425 [0.0309, 0.0541]
Two-Stream-Concat	0.9839 [0.9815, 0.9864]	0.7296 [0.7279, 0.7312]	0.1398 [0.1293, 0.1503]
Oracle	*0.9861*	*0.7330*	*0.1609*

that with $n = 10752$ paired contexts almost any consistent difference reaches significance, which is why we report effect sizes, confidence intervals, and the oracle-normalized gap alongside the p-values rather than instead of them.

Generalization to unseen pairs. Table 4.4 repeats the ablation on the 32 structurally held-out pairs of §4.4.1, spanning 32 source ASes none of the agents trained on.

Two findings matter here. First, the scoring agents generalize structurally: rewards fall on Low-Latency (0.731 to 0.607), but the *oracle* falls with them (0.733 to 0.610), and Two-Stream-Concat's optimality gap grows only from 0.0018 to 0.0033. The absolute drop is a property of what these pairs make available, not of the policy. Adaptivity in fact rises, from 0.149 to 0.190, tracking the oracle's own rise to 0.181. Second, the flat agent does not generalize at all: its Throughput reward goes *negative*, from 0.333 to −0.085. Its fixed, index-addressed action space is tied to the pairs it trained on, which is direct evidence for abandoning it (§4.1.1).

4.4.3 Intent Alignment

Adaptivity establishes that the conditioned policy changes its choice with intent; this section shows that it changes it in the *right* direction. We evaluate the single trained checkpoint under each intent and measure both the reward it earns and the metrics of the paths it selects.

Figure 4.1 and Table 4.5 report the reward $R(i, j)$ obtained when the agent is told intent i but its choice is scored under objective j. Reading down a column asks: to do well under this objective, which intent should the agent be given? The answer is the matching one in all four columns, which is the Intent Alignment property of §3.2. For Throughput the margin is decisive – being told "Throughput" scores 0.986, against 0.734 when told Low-Latency. For Low-Latency the matched intent again wins, and the mismatch penalty is largest relative to the column's range, precisely because latency is the objective that most demands re-ranking.

Table 4.4: The same ablation on 32 pairs across 32 distinct source ASes, none visited during training. Absolute rewards fall on Low-Latency, but so does the oracle's, so the drop reflects these pairs' available headroom rather than a generalization failure. The flat agent is the exception: its Throughput reward goes negative.

Method	Thpt.	Low-Lat.	Low-Loss	Bal.	Adapt.
Flat DQN	−0.085	0.517	0.652	0.246	0.008
Scoring DQN (uncond.)	0.978	0.557	0.887	0.857	0.010
Value-Concat	0.980	0.592	0.888	0.854	0.070
Two-Stream-Concat	**0.984**	**0.607**	**0.889**	**0.857**	**0.190**
Oracle	*0.985*	*0.610*	*0.892*	*0.858*	*0.181*

Table 4.5: The reward matrix behind Fig. 4.1: mean reward when the policy is told the row intent and scored under the column objective, over 10752 contexts. Column maxima in bold; the diagonal attains all four. Note the very different column ranges, which the figure's column-relative coloring deliberately hides.

	Scored under			
Told	Throughput	Low-Latency	Low-Loss	Balanced
Throughput	**0.9860**	0.6549	0.9333	0.8949
Low-Latency	0.7341	**0.7312**	0.9293	0.7824
Low-Loss	0.9478	0.7010	**0.9398**	0.8877
Balanced	0.9781	0.6786	0.9371	**0.8974**
Column range	0.252	0.076	0.011	0.115

The margins on the remaining two columns are small, and we report that plainly: the Low-Loss column spans 0.011 across all four intents, so while the diagonal wins it, no reader should take that as strong evidence. The Balanced column spans more (0.115), but almost all of it comes from the Low-Latency intent scoring badly there rather than from the other intents separating.

Why the loss axis is degenerate, and why better profiles would not fix it. The reason Low-Loss discriminates so weakly is not a poor choice of profile but the environment: across all 43008 selections, **98.85%** of chosen paths have exactly zero loss, so there is almost nothing for a loss-averse intent to avoid. Consistently, the latency-optimal and loss-optimal paths coincide in 82.8% of contexts, whereas a genuine bandwidth/latency conflict exists in **59%**. It is tempting to propose replacing Low-Loss with a profile whose optimum conflicts with high bandwidth – a hop-count-averse or trust-dominant intent – but the reward function of Eq. (4.2)

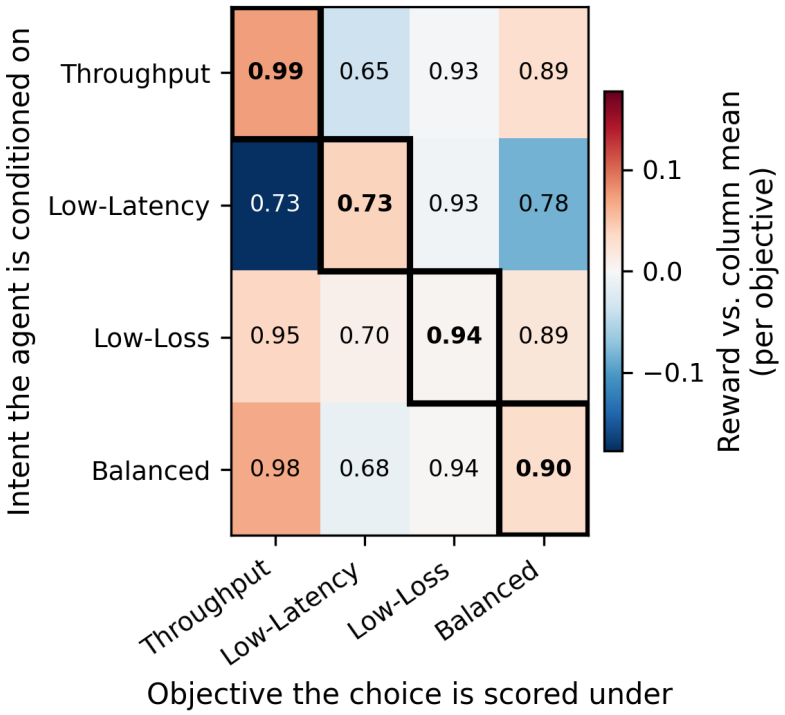

Figure 4.1: Reward matrix R(intent told, objective scored) for the conditioned policy. Cell color is the reward relative to its column mean, so a red diagonal means that conditioning on an intent is the best way to score well under that same objective. The diagonal wins every column; the margin is large on the two objectives that conflict with the high-bandwidth default and small on the two that do not. Table 4.5 gives the underlying numbers.

exposes only three levers, and hop count enters solely through the probe charge. We tested two such candidates against the *oracle*, which bounds what any policy could achieve: a `probe_extreme` profile at sixteen times the `probe_averse` weight moves the oracle's chosen hop count by 0.009 hops, and a pure loss-aversion profile with the latency term removed collapses onto the throughput optimum, matching it to within 2 Mbit/s of goodput. Since the oracle bounds every policy, no reweighting of this reward can produce a fourth discriminating intent. The honest statement is therefore that *this environment offers one genuine behavioral trade-off axis, goodput against latency*, that intent alignment is demonstrated on it, and that making loss or hop count discriminating requires changing the reward or the environment – adding an explicit hop-count term, or raising offered load until loss is common, given that only 7.8% of links are oversubscribed today. We return to this in Chapter 8.

Figure 4.2 shows the same behavior in the physical metrics of the chosen paths. Holding the decision contexts fixed and varying only the intent, the

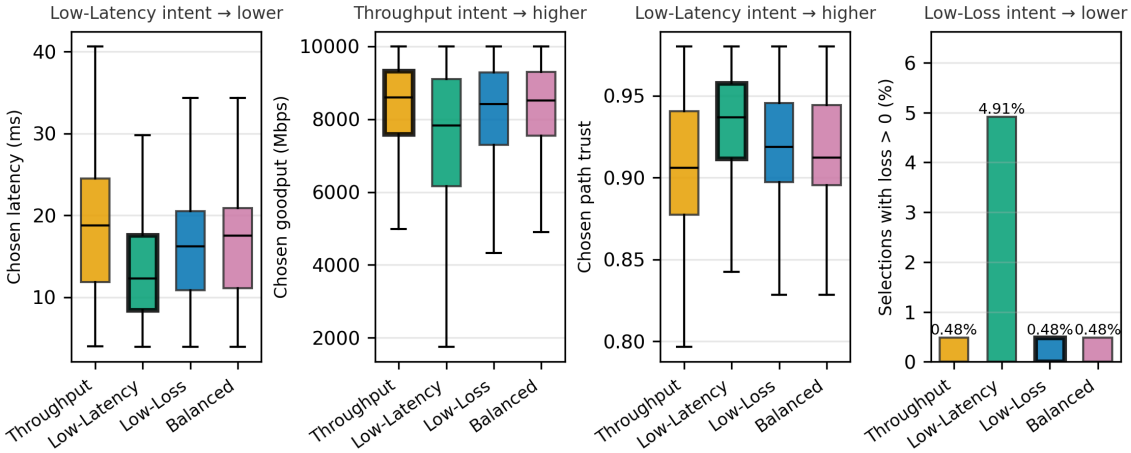

Figure 4.2: Distributions of the chosen path's metrics as the conditioned policy is told each intent, over a fixed set of decision contexts. The Low-Latency intent lowers the chosen latency and the Throughput intent raises the chosen goodput. Trust is scored with the canonical weights of Eq. (4.1) and is therefore maximized by the Low-Latency intent, not the Low-Loss one; the fourth panel reports loss exposure directly, since a boxplot of loss rate is a flat line at zero.

Low-Latency intent shifts the chosen-latency distribution down markedly (mean 12.73 ms against 18.40 ms under Throughput and roughly 16 ms under the other two), and pays for it in goodput – 7.25 Gbit/s on average against 8.24 Gbit/s. The Throughput intent shifts the chosen-goodput distribution up, though only slightly, because three of the four intents already converge on the widest paths.

The trust panel does *not* behave as the intent labels would suggest, and the discrepancy is instructive rather than a defect. The highest-trust selections come from the *Low-Latency* intent (mean 0.934), not the Low-Loss intent (0.922, barely above Balanced's 0.915 and Throughput's 0.908). This follows directly from Eq. (4.1): we score trust with canonical weights that penalize loss and delay equally, so the short paths a latency-averse intent selects maximize it, whereas the Low-Loss profile weights loss heavily and delay barely and therefore optimizes a quantity the canonical trust score largely ignores. The fourth panel makes the point directly: the Low-Latency intent is the only one that moves loss exposure at all, and it moves it *up*, to 4.91% of selections incurring any loss against the 0.48% floor that the other three intents all sit on. Buying 5.7 ms of latency means accepting shorter, more heavily loaded paths. The mechanism works; it is the loss axis that has nothing to give.

4.4.4 Zero-Shot Generalization to Unseen Intents

Every intent evaluated so far is one of the six profiles the agent trained on. That establishes only that a single network can store six behaviors – not that it has

learned a *mapping* from the intent space to behavior. This section tests the stronger property that §3.2 requires: presented with a weight vector it has never seen, does the policy behave in the correspondingly interpolated way, without retraining?

We sweep the intent along the segment between the two profiles that genuinely conflict,

$$\mathbf{w}(t) = (1-t)\cdot\mathbf{w}_{\text{Throughput}} + t\cdot\mathbf{w}_{\text{Low-Latency}}, \qquad t \in \{-0.2, -0.1, 0.0, 0.1, \ldots, 1.1, 1.2\}, \tag{4.5}$$

and evaluate each of the 15 resulting weight vectors on the same 10752 held-out contexts. Only $t = 0$ and $t = 1$ are vectors any agent has ever seen; the eleven interior points are unseen interpolations and the two outside $[0, 1]$ are extrapolations beyond both trained profiles.

Figure 4.3 shows the result, and it is unambiguous. The conditioned policy's mean chosen latency falls monotonically from 18.40 ms at $t = 0$ to 12.73 ms at $t = 1$, with a Spearman rank correlation against t of exactly -1.000. Monotonicity alone is a weak test, so we also ask how the change is distributed: no single step of the sweep carries more than 24.7% of the total span. A policy that had memorized six points would instead sit at one endpoint behavior until the weight vector crossed some threshold and then jump, concentrating essentially the whole span in one step. The endpoints reproduce the trained-profile behavior exactly, matching the Throughput and Low-Latency rows of §4.4.3, and the extrapolated points continue the trend rather than diverging – 12.05 ms at $t = 1.2$, beyond any trained intent.

Two further observations. First, the same sweep on the 32 structurally held-out pairs of §4.4.1 reproduces the result: 24.76 ms to 19.16 ms, again monotone with $\rho = -1.000$. This is *unseen intents on unseen pairs simultaneously*, and it is the strongest generalization evidence in the chapter. Second, the Value-Concat control interpolates smoothly as well, but over a span of only 2.0 ms against Two-Stream-Concat's 5.7 ms. This is Proposition 4.1 showing up from a third direction: a policy whose conditioning cannot re-rank directly can still shift behavior through the indirect effects of training, but it covers less than half as much of the behavior space.

We therefore claim zero-shot generalization in a specific and bounded sense: interpolation, and modest extrapolation, along the segment between two trained profiles, on one topology. We have not tested weight vectors far outside the trained convex hull, nor a second topology.

4.4.5 Variable Path Sets

The scoring architecture of §4.1.4 was adopted so that one policy could serve pairs with any number of candidate paths, in any order, without padding or retraining. So far that claim rests on the architecture's construction. This section measures it.

Binning the held-out decisions by their natural candidate count is uninformative here, because 7988 of the 8010 routable pairs have exactly 30 paths – the topology simply does not vary N. We therefore restrict the candidate set *at decision time* to

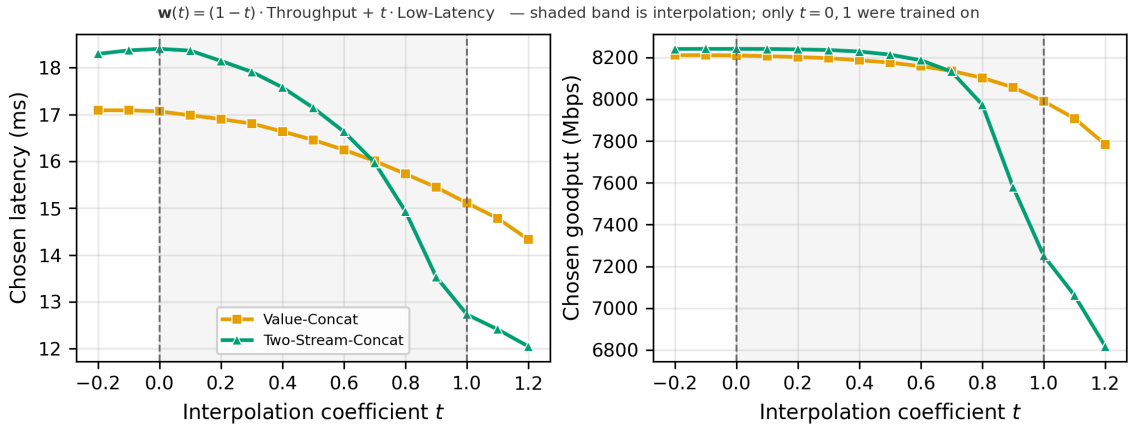

Figure 4.3: Behavior of the conditioned policies along the unseen intent segment of Eq. (4.5). Only the two dashed verticals were trained on; the shaded band is interpolation and the flanks are extrapolation. Chosen latency falls monotonically and smoothly, which is the signature of a learned mapping rather than of memorized endpoints. Value-Concat interpolates smoothly too, but traverses less than half as much behavior, as Proposition 4.1 predicts.

$N \in \{2, 4, 8, 16, 30\}$, presenting every method the identical subsets, and measure each method's regret against the best path available within that restricted set. This is a harder test than natural variation would give, since the agents trained exclusively on 30-candidate sets and every smaller value of N is off-distribution.

Figure 4.4 reports the result. Every scoring agent holds mean regret between 0.0001 and 0.013 across the whole range, while the flat DQN sits between 0.24 and 0.50 – two orders of magnitude worse, and worst in the middle of its range, where the padded action head is least well matched to the number of real alternatives. One model, no retraining, no padding, across a 15-fold change in the action count.

One caveat is worth stating rather than smoothing over: the scoring agents are slightly *better* at $N = 30$ (0.0001–0.004) than at small N (0.005–0.013). They trained only on 30-candidate sets, and the per-path bandwidth-ratio feature is normalized by the maximum within the presented set, so subsampling shifts the observation distribution off what they saw. The effect is real but bounded at about 0.01 reward: the architecture handles unseen N gracefully rather than perfectly.

Order invariance. The second half of the property is that the choice must not depend on the order in which candidates happen to be enumerated. We re-scored each of the 10752 contexts under three random permutations of the path ordering, 32256 trials per agent, and checked whether each agent selected the same *underlying path*. All four scoring and conditional agents did so in **100.0000%** of trials – exact invariance, not approximate. The flat DQN is excluded by construction: its action

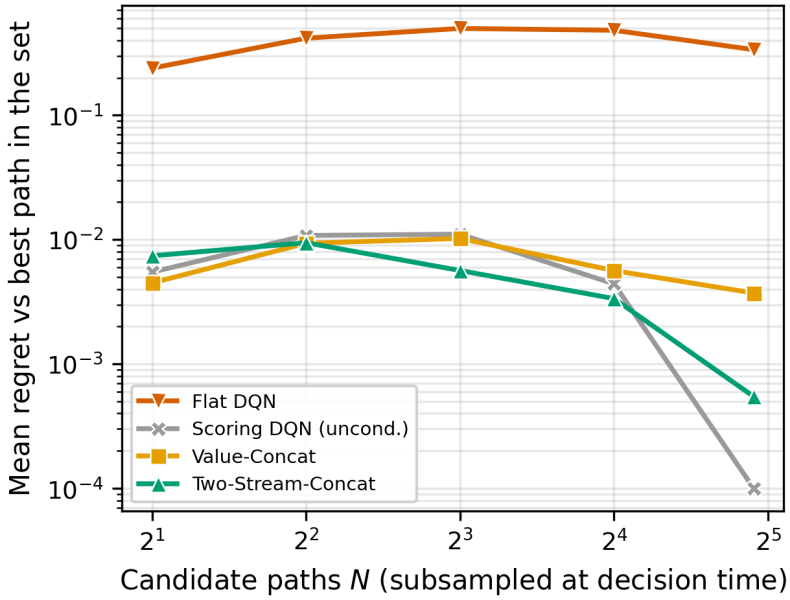

Figure 4.4: Mean regret against the best path in the presented candidate set, by set size (log scale, 10752 decisions per point, Balanced intent). One model per method, no retraining and no padding, across a 15-fold range in N. Every scoring agent stays two orders of magnitude below the flat DQN, which must address a fixed 30-way action head and degrades as N moves into the middle of its range.

is an index into a padded vector, so permuting the candidates changes what it selects, which is precisely the architectural point.

4.4.6 Probing Overhead

A path selector is only useful if its selection quality does not come at the price of measuring every path on every decision. Using the probing accounting of §4.3.3, we compare the conditioned selector against the six heuristics – first under the Balanced intent, where goodput and reward are directly comparable, then under all four.

Figure 4.5 places each method in the quality–cost plane. Widest-path attains the highest goodput (8.24 Gbit/s) but must issue a full bandwidth probe on all 30 candidate paths, at 5388 ms of probing per selection. The learned selector reaches 8.21 Gbit/s – within 0.5% of widest-path – while inspecting only the single path it selects, at 137 ms per selection: a reduction in probing cost of roughly 39× at matched goodput, and *one probe per decision rather than thirty*.

The learned selector is in fact the cheapest method in the comparison, not merely cheaper than the strongest one. Every load-blind heuristic needs its metric on all 30 candidates before it can apply its rule, so each pays 360 ms of latency probing

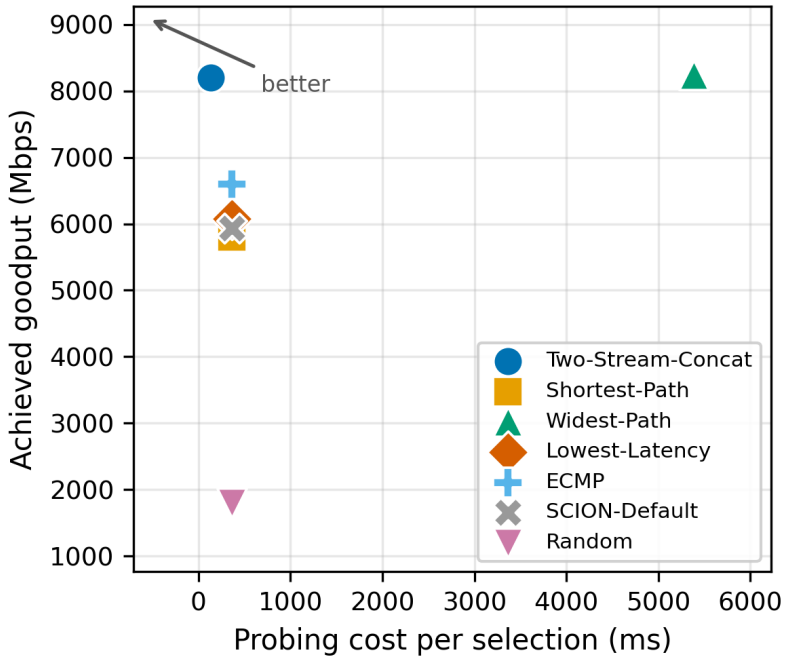

Figure 4.5: Achieved goodput against probing cost per selection under the Balanced intent (top-left is better). The learned selector reaches the goodput of exhaustive widest-path selection at roughly 40× less probing, because it inspects the one path it selects rather than all 30. It is also the cheapest method on the plot: every load-blind heuristic must measure all 30 candidates to apply its rule, and so probes about 2.6× more than the learned selector while choosing worse.

per selection – about 2.6× the learned selector's cost – and then chooses worse: ECMP, the strongest of them, reaches only 6.6 Gbit/s, and the SCION default, shortest-path, and lowest-latency deliver 5.8 – 6.1 Gbit/s, roughly 20–40% less goodput, because none of them inspects bandwidth.

The strongest heuristic changes with the intent. Running the comparison only under Balanced would understate the result, because Balanced is the one intent for which a single heuristic – widest-path – is close to the right rule. Table 4.6 repeats it under all four intents on the same contexts.

The pattern is the chapter's strongest practical result. One conditioned policy *beats every heuristic* on three of the four intents – Throughput (0.986 against widest-path's 0.982), Low-Loss (0.940 against 0.930), and Balanced (0.897 against 0.891) – and on the fourth it is within 0.0015 of lowest-latency, the heuristic purpose-built for that objective. Meanwhile each heuristic collapses under the other's objective: widest-path falls to 0.652 under Low-Latency, and lowest-latency to 0.449 under Throughput. Selecting the right heuristic per intent would require knowing the

Table 4.6: Mean intent-weighted reward by method and intent, with measurement cost, over the same 10752 contexts. Column maxima in bold. No heuristic is strong under more than one objective: each collapses under the other's. The learned selector is the only method that is at or near the top of every column, and it is the cheapest method in both probes and milliseconds.

	Measurement cost		Mean reward by intent			
Method	Probes	ms/sel.	Thpt.	Low-Lat.	Low-Loss	Bal.
Two-Stream-Concat	**1**	**132–139**	**0.986**	0.731	**0.940**	**0.897**
Widest-Path	30	5388	0.982	0.652	0.930	0.891
Lowest-Latency	30	360	0.449	**0.733**	0.899	0.629
ECMP	30	360	0.577	0.711	0.912	0.693
SCION-Default	30	360	0.417	0.726	0.893	0.610
Shortest-Path	30	360	0.383	0.721	0.888	0.590
Random	30	360	−0.569	0.247	0.558	−0.088

intent *and* switching mechanism; the learned selector absorbs both into one policy, at one probe instead of thirty and at a lower absolute measurement cost than any of them. For the conditioned selector we therefore recover, and extend across intents, the high-quality-at-low-probing property of Bourak et al. [3].

4.5 The Single-Path Performance Ceiling

The results so far show that one conditioned policy can serve the evaluated intents at low probing cost. This final experiment asks the question that motivates the next chapter: how far does the best single-path choice carry an application as the network fills up? We bin every held-out decision into six equal-count bins by its realized congestion – the mean utilization across the flow's candidate paths at that hour – and plot the goodput each method achieves against it (Fig. 4.6). Each bin holds 1792 decisions per method. Because the environment allows links to be oversubscribed, capping utilization at 1.5 rather than 1.0 (§4.1.2), the upper three bins sit above 1.0: they are hours in which the aggregate demand offered to a flow's candidate paths exceeds their capacity.

Two effects appear in the goodput panel. First, the learned selector tracks the widest-path ceiling across the entire congestion range, confirming that it makes near-optimal use of whatever single-path capacity exists, while the load-blind heuristics fall away sharply – shortest-path, lowest-latency, and the SCION default lose roughly 55–58% of their goodput from the lightest to the heaviest congestion bin, ECMP degrades more gracefully but still loses about 39%, and random collapses entirely. Second, and crucially, *the ceiling itself descends*: both

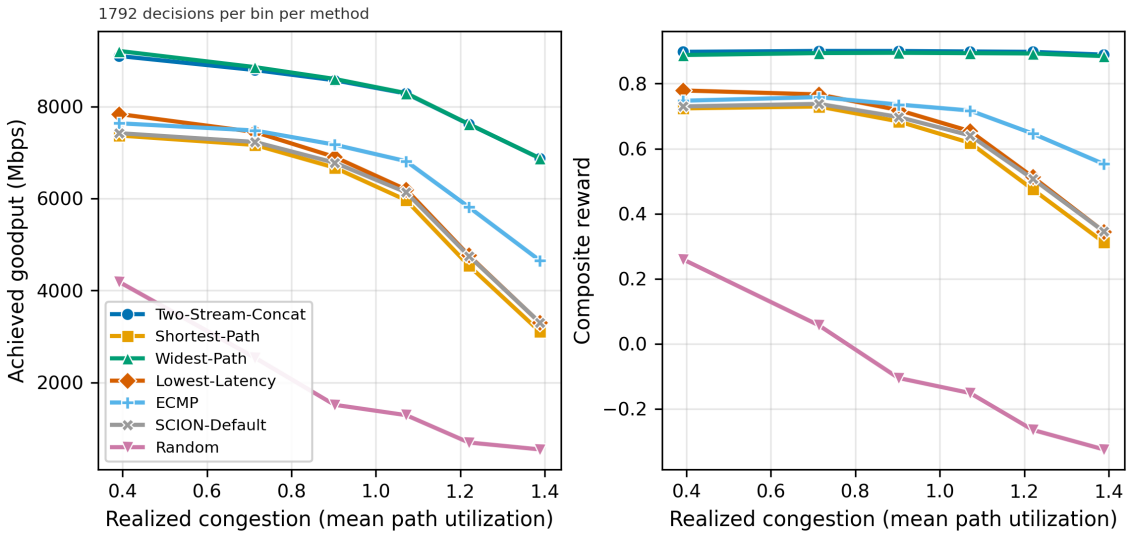

Figure 4.6: Achieved goodput (left) and intent-weighted reward (right) against realized congestion, 1792 decisions per bin per method. The learned selector tracks the widest-path ceiling at every load, but that ceiling itself falls by about 25% from light to heavy congestion: past a point, no single path can carry the demand. The right panel is the bridging argument of this thesis in one picture – the selector's *reward* stays flat while its goodput descends, because the reward measures selection quality and is blind to the ceiling by construction. The policy is doing its job; the action space is the limit.

the learned selector and widest-path drop from about 9.1 Gbit/s at light load to about 6.9 Gbit/s at heavy load, a loss of 24%. This is the single-path performance ceiling. When the shared bottleneck is congested, the best available path is still one path, and one path can carry only so much; no amount of intent-aware selection recovers capacity that a single path cannot provide. Escaping this ceiling requires spreading a flow across several paths at once – the problem Chapter 5 takes up.

Why the reward does not see the ceiling. The right panel of Fig. 4.6 tells the complementary story, and it is worth stating explicitly because a reader who takes the reward as an absolute performance measure will find the two panels contradictory. The learned selector's reward is essentially flat across the congestion range, 0.898 to 0.889, while its goodput falls by a quarter. This is a direct consequence of Eq. (4.2), in which goodput is normalized against the best candidate path at the same hour (§4.1.3): a policy that keeps choosing the best available path scores near the top whatever that path can carry. The heuristics, which do not keep choosing the best available path, collapse on reward as well – shortest-path from 0.72 to 0.31, and the fixed-random baseline from 0.26 to −0.32. The ceiling is therefore invisible to the objective the agent optimizes, which is precisely why

breaking it takes a change of *action space* rather than a better policy or a better reward.

Summary. This chapter established four claims. First, a fixed-objective, fixed-action DQN is the wrong shape for this problem: replacing it with a permutation-equivariant per-path scorer raises goodput by 41% over five seeds, from 5.81 Gbit/s to 8.21 Gbit/s with disjoint confidence intervals, gives exactly invariant selection under 32256 permutation trials, and holds regret flat from $N = 2$ to $N = 30$, while the flat agent's Throughput reward goes negative on pairs it never trained on. Second, making one policy serve many intents is not a matter of feeding it the intent: by Proposition 4.1, conditioning that enters only path-independent terms cancels under the arg max, and over five seeds the value-stream variant re-ranks roughly three times less than one whose conditioning reaches the per-path comparison, with disjoint confidence intervals. Third, the conditioned policy generalizes over the intent *space* rather than merely across trained points: behavior interpolates monotonically between trained intents ($\rho = -1.000$) and does so on pairs never seen in training. Fourth, it does this within 0.25% of a per-context reward oracle on every evaluated intent, beating every heuristic on three of four intents while issuing one probe instead of thirty.

Two scoping caveats belong with these claims. This environment offers exactly one genuine behavioral trade-off axis – goodput against latency – so intent alignment is demonstrated there rather than across all four evaluated profiles, and §4.4.3 shows why no reweighting of this reward could change that. And every result rests on a single topology. What survives both caveats is the chapter's final finding: even a near-optimal single-path selector loses about a quarter of its goodput under heavy congestion, because a single path is a hard capacity bound – the ceiling that Chapter 5 sets out to break.

5 Joint Application–Network Optimization over Multipath

Chapter 4 chose a single path for a flow and found its ceiling: when the shared bottleneck saturates, even the best single path cannot carry the demand. This chapter starts from a more ambitious premise. SCION offers not one path but several, so why commit to one? And if Mortise [16] can tune a *transport mechanism* to the application, why not tune the *application itself* in the same breath? We design and evaluate a system that (i) uses multiple paths at once via Multipath QUIC [5] and (ii) jointly optimizes what the application sends – the video encoder's target bitrate – and how the network spreads it – the per-path traffic split – to maximize the quality of experience of a real-time video call. Section 5.1 presents the design, §5.2 the testbed, and §5.3 the evaluation, whose central finding is that the value of learned multipath scheduling is a direct function of network volatility.

5.1 System Design: Co-Optimizing Rate and Traffic Split

Section 5.1.1 motivates the joint, multipath framing and the strawmen it improves on. Section 5.1.2 presents the two-timescale hierarchical agent pair. Section 5.1.3 specifies the observations, actions, and rewards. Section 5.1.4 explains the coupling that keeps the two agents cooperating rather than working at cross purposes. Section 5.1.5 extends the scoring idea of Chapter 4 from a discrete choice to a continuous split over a variable, possibly changing, path set.

5.1.1 Motivation and Strawmen

Interactive video is a demanding, ubiquitous workload with no playback buffer to hide variance: a frame that misses its deadline is simply lost. Its QoE depends on two decisions that are usually made in isolation. The *application* chooses an encoding bitrate – too high and the network drops frames, too low and quality suffers. The *network* chooses how to carry the resulting bytes – over one path or spread across several. Optimizing either alone is a strawman:

Application-only. Adapt bitrate to the aggregate network but carry it over a fixed or naive path assignment. This wastes SCION's path diversity and, as our results show, collapses when the paths it depends on degrade.

Network-only. Schedule cleverly across paths but treat the offered load as exogenous. The scheduler cannot relieve congestion it is itself creating by admitting too high a bitrate.

The two decisions are coupled through the same bottleneck: bitrate sets the load the scheduler must place, and the scheduler's success sets the effective capacity the bitrate controller sees. Optimizing them jointly is the design's premise. The challenge is that they naturally operate on *different timescales* – an encoder should not change bitrate every frame, but a scheduler must react per frame – which motivates a hierarchy rather than a single monolithic agent.

5.1.2 Two-Timescale Hierarchical Agents

We split control into two cooperating agents that act at different cadences, a "brain and body" decomposition.

Application agent (slow). Every 1 s (once per 30 frames at 30 fps) it observes an aggregate summary of the connection and outputs a single target encoder bitrate in the range 300 – 6000 kbit/s. It is deliberately *horizon-agnostic* – it never observes how far along the call is – so the same policy applies to a call of any length, an essential property for a controller meant to run in unbounded real sessions.

Transport agent (fast). Every frame (≈33 ms) it observes per-path transport state *and the application agent's current target bitrate*, and outputs a split of that frame's bytes across the available paths.

Both are trained with soft actor-critic (§2.3), whose continuous action support and sample efficiency suit these controls and the cost of simulator rollouts.

5.1.3 Observations, Actions, and Rewards

Application observation and action. The application agent sees a small fixed-size vector: normalized current bitrate, smoothed RTT, jitter, loss, and throughput. Its action is one continuous scalar in $[-1, 1]$, mapped affinely to a target bitrate in 300 – 6000 kbit/s.

Transport observation and action. The transport agent sees a global block (target bitrate, RTT, loss, throughput) and, for each path, a feature vector (congestion window, smoothed RTT, buffer occupancy, per-path throughput, per-path loss). Its action is an N-vector passed through a softmax to yield a split $(w_1, \ldots, w_N)$ that is non-negative and sums to one – a valid allocation by construction.

Rewards. The rewards encode the QoE objective and the division of responsibility between the agents. The application's per-window QoE reward composes a quality term with penalties:

$$\text{QoE} = a \cdot \frac{\text{VMAF}(\cdot)}{100} - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}} - d \cdot \text{loss}, \qquad \text{clipped to } [-2, 1], \tag{5.1}$$

where $\widetilde{\text{lat}}$ and $\widetilde{\text{jit}}$ are latency and jitter normalized (by 200 ms and 50 ms) and clipped, and the default weights are $(a, b, c, d) = (1, 0.5, 0.5, 1)$. The quality term uses a VMAF surrogate (§5.2); when a learned surrogate that already accounts for loss is used, the explicit loss penalty is dropped to avoid double-counting. The transport agent receives a per-frame reward that credits delivery without quality, since it does not control bitrate:

$$R_{\text{transport}} = (1 - \text{loss}) - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}}, \qquad \text{clipped to } [-2, 1]. \tag{5.2}$$

5.1.4 Coupling the Two Agents

The design's subtlety is keeping the agents cooperative. Two mechanisms couple them.

Observation coupling. The transport agent observes the application's current target bitrate. It therefore schedules *knowing* how much load it must place, and can, for instance, spread an aggressive bitrate across more paths rather than overrunning one. This is what makes the pair hierarchical rather than two agents optimizing in ignorance of each other.

Reward decomposition. The application owns quality (the VMAF term) and is credited for the latency, jitter, and loss experienced over the window its bitrate governed; the transport agent is credited only for how well it delivered each frame. This assigns each decision the consequences it can actually control – the encoder is not blamed for a poor split, nor the scheduler for an over-ambitious bitrate – while the shared bottleneck still couples them: a bad split shows up as latency/loss in the application's reward, and an over-high bitrate shows up as loss the transport agent cannot schedule away.

5.1.5 Continuous Scoring over a Variable Path Set

The transport agent inherits the single-path selector's problem – a variable, possibly changing number of paths – and its solution, adapted from a discrete choice to a continuous allocation. A shared per-path encoder maps each $(\text{global}, \text{path}_i)$ pair to a logit; the logits are softmaxed into the split, so the policy is permutation-equivariant and independent of the path count. To handle paths that appear and disappear, each path carries a *liveness* flag and the state includes a binary mask;

the softmax excludes dead paths, so traffic is never allocated to a path that is gone. The critic aggregates per-path embeddings with a masked mean-pool – a Deep Sets [22] operation – yielding a value estimate invariant to path ordering and count. This continuous scoring agent is the natural sibling of the discrete scoring selector of Chapter 4: the same architectural idea, once for "pick a path" and once for "divide across paths."

5.2 Implementation: Multipath Real-Time Video Testbed

Realizing the design requires a testbed that is faithful enough to expose real multipath and congestion behavior yet fast enough for reinforcement-learning rollouts. We resolve this tension with a single data-plane abstraction served by two interchangeable backends (§5.2.1): a packet-level NS-3 simulator and a fast analytical mock. Section 5.2.2 describes the real-time video and multipath model, §5.2.3 the agent implementation and the VMAF surrogate, and §5.2.4 the known gaps that scope the current results.

5.2.1 Data-Plane Abstraction and Two Backends

Both backends implement one abstract `DataPlane` interface, so the RL code, observations, and rewards are written once and run unchanged against either. This lets us iterate quickly on the mock and validate on the higher-fidelity simulator without divergence.

NS-3 backend. A packet-level simulation in which the connection's several paths are modeled as independent subflows over a topology of links, each with its own capacity, propagation delay, and active-queue management (FqCoDel). Realistic contention comes from OnOff UDP cross-traffic with exponentially distributed on/off periods, phase-shifted per path so the paths do not degrade in lockstep. Each video frame's bytes are striped across the subflows according to the transport agent's split, and delivery is tracked by a byte-watermark scheme – avoiding per-packet tagging – from which per-frame latency, jitter, and deadline-miss loss are measured. Python and NS-3 communicate through an ns3-ai shared-memory bridge.

Mock backend. A pure-Python, per-seed-deterministic model in which each path has a capacity envelope (a smooth sinusoidal component plus bursty cross-traffic and noise) and a standing-queue model that produces bufferbloat when a path is overdriven. It runs orders of magnitude faster than NS-3 and is used for rapid training and for scenarios the NS-3 body does not yet emit.

Non-stationary dynamics. The mock backend can optionally inject the volatility that makes multipath scheduling matter: *path churn* (paths appear and disappear,

and bytes sent to a dead path are lost), *regime shifts* (abrupt changes in the best path's capacity), *congestion bursts* (transient per-path capacity collapses), and *correlated failures* (shared-bottleneck groups that degrade together). These are off by default and are the conditions under which §5.3 finds the learned split decisive.

5.2.2 Real-Time Video and Multipath Model

The video source mirrors a conferencing encoder: a fixed frame rate (30 fps by default), variable frame sizes (with periodic larger I-frames and per-frame jitter), no playback buffer, and a deadline after which a late frame is discarded as lost. The aggregate target bitrate ranges over 300 – 6000 kbit/s. The canonical topology is deliberately *non-dominant*: no single path can carry full-quality video, and the aggregate usable capacity sits below the quality knee, so the system must genuinely combine paths and trade rate against delivery. For example, a four-path configuration mixes paths of differing capacity and RTT – including a higher-capacity but high-latency "trap" path – under heavy, path-specific cross-traffic, so that neither the widest nor the lowest-latency path is right on its own.

5.2.3 Agents and QoE Surrogate

Agents. Both agents are soft actor-critic learners with twin critics and automatic temperature tuning. The transport agent has two variants: a *flat* version whose actor and critic consume a fixed-length concatenation of the global and per-path features (a fixed path count), and a *scoring* version implementing the permutation-equivariant, mask-aware design of §5.1.5 with a shared per-path actor and a Deep Sets masked-mean-pool critic. The hierarchical training loop runs the two agents at their respective cadences, storing each agent's transitions in its own replay buffer and crediting the application agent over the window of frames its bitrate governed.

VMAF surrogate. The quality term of Eq. (5.1) needs a mapping from an encoding (and network conditions) to perceptual quality. Two are supported: a default bitrate-only logarithmic curve fit through published quality anchors, and a learned surrogate that interpolates a quality-of-service→VMAF table over bitrate, loss, delay, and jitter (produced by a sibling data-generation project). When the learned surrogate is active, the explicit loss penalty in Eq. (5.1) is dropped to avoid double-counting the loss it already reflects.

5.2.4 Known Gaps

The current testbed has three limitations that scope the results of §5.3 and define concrete follow-up work.

- **Dynamics parity.** The non-stationary dynamics (churn, regime shifts, bursts) are currently emitted only by the mock backend; porting them into the NS-3 body – adding per-path liveness to the simulation – is required before the packet-level and analytical results can be compared on equal footing. **[TODO:** *report NS-3 vs. mock parity once the C++ dynamics land.*]

- **Transport realism.** Subflows are modeled with TCP-like congestion control rather than true Multipath QUIC; this preserves congestion, queueing, and loss dynamics but masks some network loss as latency. Swapping in a real QUIC module is future work.

- **Surrogate coverage.** The learned VMAF surrogate's loss, delay, and jitter axes are currently under-sampled, so the quality term is effectively bitrate-dominated; a richer surrogate would activate those axes and let quality respond to delivery conditions directly.

5.3 Experimental Evaluation: Regimes of Value

This section evaluates the multipath system. Section 5.3.1 gives the setup and baselines; §5.3.2 reports the central finding – that the value of learned multipath scheduling depends sharply on how volatile the network is; and §5.3.3 isolates the contributions of the joint design and the scoring transport agent. The numbers reported here are from preliminary runs on the mock backend; packet-level NS-3 confirmation is pending (§5.2.4) and is marked where relevant.

5.3.1 Setup and Baselines

Scenarios. We use two contrasting scenarios. The *static* scenario is the non-dominant four-path topology of §5.2.2, with stationary (if heavily loaded) paths. The *dynamic* scenario is a six-path setting with the non-stationary dynamics of §5.2.1 enabled – path churn, regime shifts, and congestion bursts – so that the identity and quality of the best paths change during a call.

Metric. The primary metric is mean per-window QoE (Eq. (5.1)); we also report the underlying achieved bitrate, latency, loss, and VMAF.

Baselines. The learned pair (application agent + transport agent) is compared against fixed scheduling policies driving the *same* learned bitrate controller where applicable: *even* split across live paths, *single* best-active path, and *proportional* allocation by capacity. The decisive ablation is *app-only*: the learned bitrate controller with a naive even split, isolating the value of the learned transport agent.

5.3.2 Results: When Does Splitting Help?

The headline result is that the learned split's value is conditional on volatility. Table 5.1 summarizes preliminary QoE across scenarios and methods.

Two observations stand out. First, in the **static** scenario the learned pair (0.633) beats the single-best-path baseline (0.514) by roughly 23%, but the app-only ablation (0.665) is actually on par with – even marginally above – the full system. When paths are stable, a good bitrate controller over a simple split captures nearly all the achievable QoE, and the transport agent has little left to earn. A practitioner reading only the static numbers might reasonably conclude that learned scheduling is not worth its complexity.

Second, the **dynamic** scenario overturns that conclusion. The learned pair (0.606) again leads, but now the app-only ablation *collapses to* −0.212: with paths churning and bursting, a fixed even split routes traffic onto dead or congested paths, driving latency and loss up despite a well-chosen bitrate. The fixed-schedule baselines degrade in order of how much they react to path state – single-best (0.402), proportional (0.209), even (0.109) – and only the learned transport agent, which observes path liveness and reallocates, sustains high QoE. This is the condition under which learned multipath scheduling is not a marginal optimization but the difference between a usable and an unusable call.

[TODO: *add variance/confidence intervals over seeds, and a time series from a representative dynamic episode showing the transport agent reallocating away from a path as it churns or bursts (the* `dyn-scoring` *run illustrates convergence from strongly negative early-episode reward to stable positive QoE with latency in the tens of milliseconds and loss a few percent).***]**

Placeholder — Figure: QoE over training episodes for the learned pair vs. baselines in the dynamic scenario, and a within-episode trace of the per-path split tracking path liveness.

5.3.3 Ablation Study

Joint vs. app-only. Table 5.1 already provides the core ablation: removing the learned transport agent (app-only) is nearly free when the network is stable but catastrophic when it is volatile. This quantifies the design premise of §5.1.1 – the two decisions must be co-optimized precisely in the regimes where the network is hard.

Flat vs. scoring transport. We compare the flat transport agent (fixed path count) against the permutation-equivariant scoring agent (§5.1.5) on the dynamic scenario, where the number of live paths changes. **[TODO:** *report the flat-vs-scoring*

Table 5.1: Preliminary mean QoE (Eq. (5.1)) by scenario and method (mock backend). Higher is better. In the static scenario the split barely matters – the learned transport agent adds little over app-only – whereas in the dynamic scenario the split is decisive and app-only collapses.

Method	Static 4-path	Dynamic 6-path
Learned pair (app + transport)	**0.633**	**0.606**
App-only (learned rate, even split)	0.665	−0.212
Single best-active path	0.514	0.402
Proportional (by capacity)	**[TODO:** *fill*]	0.209
Even split	**[TODO:** *fill*]	0.109

comparison; the expected finding is that only the scoring/mask-aware agent handles a changing path count gracefully, since the flat agent cannot represent appearing/disappearing paths.]

Backend parity. All numbers above are from the mock backend. Confirming them on the packet-level NS-3 backend requires the dynamics port of §5.2.4. [**TODO:** *report NS-3 results for the static scenario now, and for the dynamic scenario once the C++ dynamics land, to establish that the mock findings transfer to packet-level simulation.*]

5.3.4 Summary

Preliminary results support the joint, multipath design: co-optimizing bitrate and per-path split matches or beats every fixed-scheduling baseline, and the advantage of learned scheduling grows from negligible in a stable network to decisive under churn and bursts. The main open items are seed-level statistics, the flat-vs-scoring comparison, and NS-3 confirmation.

6 Discussion and System Synthesis

This chapter steps back from the two systems to their joint meaning. Section 6.1 synthesizes what intent-conditioned single-path selection and joint multipath co-optimization say together about intent-driven network control. Section 6.2 then analyzes both systems along the axes that decide whether learned path control is practical – overhead, scalability, and generalization – and §6.3 states the deployment gaps and trust assumptions under which the controllers operate. Where a claim depends on measurements still in progress, it is marked.

6.1 Synthesis of Findings

Read together, the two systems tell a consistent story about *when* additional control machinery earns its keep.

6.1.1 Interpretation

Intent conditioning turns a family of policies into one. The central lesson of Chapter 4 is that the right way to serve many objectives is not many models but one model that takes the objective as input – provided the objective reaches the part of the network that actually decides. Proposition 4.1 is the crisp version of this point: a conditioning signal that enters identically across the alternatives cancels under the comparison that picks the action, so the policy can look conditioned while behaving identically. Supplying the objective to the per-path terms avoids this, because the objective can then interact with each candidate's own feature values and change which one wins. This is a design principle, not a SCION-specific trick: any comparison-based selector conditioned on an objective must route that objective into its per-alternative terms.

The value of multipath control is a function of volatility. The central lesson of Chapter 5 is that learned multipath scheduling ranges from nearly worthless to decisive depending on the network. In the static scenario, a good bitrate controller over a trivial split captured essentially all achievable QoE, and the learned transport agent added little (§5.3.2). In the dynamic scenario, the same app-only configuration collapsed to negative QoE while the learned split sustained a usable call. The practical implication is that one should not evaluate multipath schedulers only in stable conditions – that is precisely the regime in which they

look unnecessary – and that the case for learned scheduling rests on volatility, churn, and bursts, where static policies cannot adapt.

One principle, two layers. Both systems instantiate the Mortise principle [16] – intent as a first-class input that reshapes a network mechanism – at successively deeper points of control. The single-path selector reshapes *which path* is chosen; the multipath system reshapes both *how the paths are used* and *what the application produces*. The progression suggests a general recipe: expose the application's intent, and let it steer every controllable decision between the application and the wire.

6.1.2 Implications

The results matter for how we build controllers on path-aware architectures. Prior learned path selectors [3] and learned congestion or scheduling controllers [9, 23, 20] each optimize a fixed objective; our work argues that on a path-aware substrate the objective should be a knob, and that a single conditioned policy can replace a zoo of per-objective models. For real-time media specifically, the joint bitrate-and-split result reinforces a theme from learned ABR [11] – that application and network decisions are better made together – and extends it to the multipath setting that SCION makes routine. More broadly, the two systems are evidence that SCION's endpoint path choice is not merely a routing convenience but an enabler of application-aware optimization that is awkward or impossible in a destination-based Internet.

6.1.3 Limitations

The results are bounded in ways that the analysis (§6.2) details and that fairness requires restating here. Both systems are evaluated in simulation, and the multipath system's headline dynamic-scenario numbers are from the mock backend pending NS-3 confirmation (§5.2.4). The single-path study rests on one topology and one training seed, with a purely temporal train/test split, so its quantitative margins should be read as indicative rather than as established effect sizes (Sections 4.4.1 and 4.4.2). Intent in this thesis is *declared*, not *inferred*: we assume the application can state its objective, and we do not study how to recover a latent intent from traffic – a natural but separate problem. The transport model is TCP-like rather than true Multipath QUIC, and the VMAF surrogate is bitrate-dominated, so absolute QoE figures should be read as relative comparisons rather than calibrated user scores. Finally, we have not addressed online, safe learning against live traffic, which any deployment would require. None of these undercut the two qualitative findings – conditioning works when it reaches the per-alternative comparison and provably cannot when it does not, and multipath control pays off in proportion to volatility – but they do delimit how far the quantitative results should be pushed.

6.2 System Analysis

This section analyzes the two systems along the dimensions that determine whether learned path control is practical: overhead (§6.2.1), scalability (§6.2.2), and generalization (§6.2.3).

6.2.1 Overhead

Two kinds of overhead matter: measurement and computation.

Measurement (probing) overhead. In the single-path selector (Chapter 4), probing is a first-class cost charged in the reward (Eq. (4.2)) and accounted per decision (§4.3.3). The learned selector probes only the paths it inspects, in contrast to heuristics that require a metric on every path, so its measurement footprint is a fraction of an exhaustive scan at comparable selection quality – and, in fact, smaller than that of every heuristic baseline, since each of those must measure all candidates before applying its rule (§4.4.6). In the multipath system (Chapter 5), per-path state is derived from ordinary transport feedback (RTT samples, acknowledgements, loss signals) rather than dedicated probes, so the marginal measurement cost of the split decision is low.

Computational overhead. Both policies are small neural networks evaluated once per decision. The single-path selector runs at path-selection granularity (seconds to minutes) and is comfortably cheap. The multipath transport agent is the tighter constraint: it runs *per frame* (≈33 ms at 30 fps), so its inference must fit well inside a frame interval. The scoring architecture keeps this affordable – a shared per-path encoder evaluated N times and a softmax – but the per-frame budget is the binding real-time constraint. For the single-path selector the margin is ample: timing only the decision itself, over 1280 held-out contexts of 30 candidate paths each on a 22-core CPU host with no GPU, the conditioned selector decides in under 0.3 ms on average and under 2 ms at the 99th percentile across repeated runs. That is three orders of magnitude inside a path-selection budget of seconds to minutes, and comfortably inside a 33 ms video frame. **[TODO:** *report the corresponding measurement for the multipath transport agent, whose per-frame budget is the tighter constraint.***]**

6.2.2 Scalability

The scoring architecture is what makes both systems scale in the number of paths. Because a shared network scores each path independently and decisions are made by comparison (§4.1.4) or softmax (§5.1.5), the parameter count is independent of N, and inference is linear in N. This lets one trained model serve source–destination pairs with any number of paths and topologies of any size, without

retraining or padding to a fixed maximum – the limitation of the flat agents. The remaining scalability question is training coverage: the environment must expose enough variety of path counts, topologies, and conditions for the shared scorer to generalize. **[TODO:** *report how selection quality holds as topology size and path count grow, ideally on topologies larger than those seen in training.*]

6.2.3 Generalization

Generalization is required along three axes. **Across intents:** the conditioning of Chapter 4 is meant to interpolate to intents between (and, ideally, beyond) the trained reward profiles, so that a novel weight vector yields sensible behavior without retraining. Section 4.4.4 confirms it: sweeping the intent along the segment between two trained profiles moves the chosen path's latency monotonically, with a Spearman correlation of −1.000 and no single step carrying more than a quarter of the span, and modest extrapolation beyond both endpoints continues the trend. **Across topologies and path counts:** the permutation-equivariant design gives structural generalization, and §4.4.5 confirms both halves of it empirically – exact order invariance over 32256 permutation trials, and regret held two orders of magnitude below the fixed-action agent across a 15-fold range in candidate count. Section 4.4.2 adds a structurally disjoint set of 32 pairs spanning 32 source ASes never visited in training, on which the conditioned selector's optimality gap grows only from 0.0018 to 0.0033 while the flat agent's Throughput reward goes negative. What remains untested is scale: all of this is one topology of 90 ASes. **Across conditions:** the dynamic scenario of Chapter 5 (§5.3.2) is itself a generalization stress test, since the policy must cope with churn and bursts rather than a stationary environment; the collapse of the fixed-schedule baselines there shows that static policies do *not* generalize across volatility, which is the gap learned control fills. For the single-path system, off-distribution degradation is bounded on two of these three axes: unseen intents cost nothing measurable (§4.4.4), and unseen pairs and candidate-set sizes cost at most about 0.01 reward (Sections 4.4.2 and 4.4.5). **[TODO:** *quantify the remaining axes – larger topologies for both systems, and unseen dynamics regimes for the multipath system – to bound how gracefully performance decays there.*]

6.3 Deployment Considerations and Trust Assumptions

6.3.1 Deployment Gaps

Several gaps still separate these prototypes from a deployment.

- **From simulation to reality.** Both systems are trained in simulation. The single-path study relies on a synthetic traffic and beaconing environment; the multipath system on NS-3 and a mock transport. Bridging the gap

to deployment would require sim-to-real transfer – for example through domain randomization or online fine-tuning against live measurements – and the packet-level NS-3 backend is the intended stepping stone (§5.2.4).

- **Real transport.** The multipath system models subflows with TCP-like congestion control rather than true Multipath QUIC [5]. A real QUIC integration is therefore needed to validate the per-frame scheduling under an actual multipath transport.
- **Online and safe learning.** A deployed agent would learn or adapt against live traffic, where exploration has real cost. Bounding worst-case behavior – for example by falling back to a safe heuristic when the policy is uncertain, or by constraining exploration – is a prerequisite that this work does not address.

Two properties help on the positive side. First, the endpoint-driven nature of SCION path selection means these controllers require no network-internal changes. Second, the horizon-agnostic application agent (§5.1.2) is designed for unbounded real sessions rather than fixed-length episodes.

6.3.2 Trust Assumptions

This thesis is a performance and control study, not a security study, so in place of a classical attacker model we state the trust assumptions under which the learned controllers operate and note where security considerations would enter a real deployment.

- **Honest measurements.** We assume that the per-path measurements an agent acts on – whether obtained by active probing (Chapter 4) or transport feedback (Chapter 5) – are honest. A malicious on-path element that fabricates favorable metrics could steer the agent toward a bad path; defending against that is out of scope, although the trust term below offers partial mitigation.
- **SCION path authenticity and trust.** SCION's cryptographically authenticated paths mean an agent knows *which* ASes a path traverses and cannot be silently redirected. The single-path reward includes a *trust* component derived from path quality, allowing the agent to prefer higher-confidence paths and discount those with anomalous loss or latency – a lightweight hedge against unreliable or adversarial path advertisement, not a full defense.
- **No adversarial control of the learner.** We assume the agent's own training and inference pipeline is trusted; adversarial manipulation of the RL policy (e.g. reward poisoning or observation attacks) is beyond this work.

A production deployment would need to revisit these assumptions – in particular by verifying measurement integrity and by bounding the damage a single misreported path can do.

7 Related Work

Where Chapter 2 covered the systems this thesis *builds on*, this chapter surveys work that attacks the *same or similar problems* and positions our contributions against it. We group it into learned routing and traffic engineering (§7.1), learned congestion control and multipath scheduling (§7.2), adaptive bitrate and QoE-driven control (§7.3), goal-conditioned and modulated policies (§7.4), and intent-based networking (§7.5).

7.1 Learned Routing and Traffic Engineering

A line of work applies reinforcement learning to routing and traffic engineering. *Learning to Route* [17] studies RL and learned representations for routing, and experience-driven networking [21] uses deep RL for traffic-engineering decisions. These target network-operator objectives (e.g. minimizing congestion globally) from a control-plane vantage point. Our single-path selector differs in both the actor and the objective: the decision is made by an *endpoint* choosing among cryptographically authenticated SCION paths, and the objective is the *application's* intent rather than a global network utility. Most directly related is the DQN SCION path selector of Bourak et al. [3], which we extend; the differences – a variable-action scoring architecture and per-path intent conditioning that removes the single-objective limitation – are the substance of Chapter 4.

7.2 Learned Congestion Control and Multipath Scheduling

Learned transport controllers include Aurora [9], which casts congestion control as RL, and Orca [1], which combines a learned layer with a classic controller. Mortise [16] is the closest in spirit: it auto-tunes congestion-control parameters to the running application's QoE, and its principle – intent as an input that reshapes a mechanism – is the template we transplant into path selection and joint multipath control. For multipath *scheduling*, ReLeS [23] learns a Multipath QUIC scheduler and Peekaboo [20] learns to schedule across heterogeneous paths in dynamic environments; both optimize a scheduling policy for a given, exogenous load. The multipath system's distinction is that it does *not* treat the offered load as fixed: it co-optimizes the application's bitrate with the split, so the controller can

relieve congestion it would otherwise create – a coupling those schedulers, and the Multipath TCP mechanisms of RFC 8684 [6], do not address.

7.3 Adaptive Bitrate and QoE-Driven Control

Learned adaptive bitrate control, exemplified by Pensieve [11], showed RL beating hand-crafted ABR heuristics for buffered video streaming, using QoE-shaped rewards and quality metrics such as VMAF [10]. Our multipath system shares the QoE-driven, learned-bitrate stance but targets *real-time* video (no playback buffer, deadline-based loss) and, crucially, couples the bitrate decision to a simultaneous multipath scheduling decision. Where Pensieve adapts rate to a single, exogenous network path, we adapt rate *and* decide how to spread it across several paths, jointly.

7.3.1 Mortise: intent-driven tuning of a transport mechanism

The conceptual template for both parts is Mortise [16]. Mortise observes that a single congestion-control algorithm cannot be optimal for every application, and instead *auto-tunes* the algorithm's parameters to maximize the running application's QoE, using the application's revealed preferences over throughput, latency, and loss to steer the tuning. The essential idea – treat application intent as a first-class input that reshapes a network mechanism rather than a constant compiled into it – is exactly what we transplant. The single-path selector applies it to *path selection* (which path to use), and the multipath system applies it to *joint rate and multipath scheduling* (what the application sends and how it is spread across paths).

7.4 Goal-Conditioned and Modulated Policies

The machinery for serving many objectives with one policy comes from goal-conditioned RL – universal value function approximators [14] and hindsight relabeling [2], which learn a value function jointly over states and goals. That literature is largely silent on *where* in a network the goal should enter, because its policies are not comparison-based. Ours are, and our contribution is a specific insight about that gap: for a comparison-based (argmax or softmax) selector, conditioning must reach the per-alternative terms, because an additive, symmetric signal cancels under the comparison (§4.2). This connects a general representation-learning idea to a concrete failure mode in networked decision-making.

7.5 Intent-Based Networking

Finally, "intent" also names a body of network-management work concerned with translating high-level operator goals into configuration, surveyed in RFC 9315 [4]. That community's "intent" is an operator's declarative policy for a whole network; ours is an *application's* per-flow QoS preference consumed by a learned endpoint controller. The two are complementary rather than competing – one could imagine operator intent bounding the space in which application intent is served – but they operate at different layers and timescales, and our techniques address the fine-grained, per-flow end of the spectrum.

8 Conclusion

This thesis asked how an endpoint on a path-aware Internet should choose the best use of its paths given the intention of the application and the state of the network. It answered in two movements that share one idea, drawn from Mortise [16]: treat application intent as a first-class input that reshapes a network mechanism, not as a constant compiled into it.

Chapter 4 turned a fixed-objective DQN path selector for SCION [3] into an intent-conditioned one. Two design decisions carried the work: a permutation-equivariant *scoring* architecture that handles a variable, changing set of candidate paths, and the routing of the intent vector into the *per-path comparison* itself, which lets a single policy serve a range of objectives – including ones it never trained on – without retraining. Chapter 5 asked what happens if we use several paths at once and optimize the application itself, and built a two-timescale hierarchical agent pair that co-optimizes video bitrate and per-path traffic split over Multipath QUIC, coupled so that each agent accounts for the other.

What we learned. The most useful findings were the ones that cut against intuition. First, that *where* intent is injected matters more than *that* it is injected: for a comparison-based selector, a conditioning signal reaching only the terms shared by every candidate leaves decisions unchanged, because it cancels under the argmax; the intent has to reach each alternative's own features to reorder them. Second, that the value of learned multipath scheduling is *not* a fixed quantity but a function of network volatility – negligible when paths are stable, where a good bitrate controller over a trivial split suffices, yet decisive under churn and bursts, where the same app-only configuration collapses. Both findings sharpen when, and when not, to reach for the more sophisticated mechanism.

Future work. Several threads remain open, and much of the draft's structure is deliberately laid out to guide them. On the experimental side: adding seed-level statistics and the flat-versus-scoring comparison for both systems, testing the single-path selector on intents and source–destination pairs it never trained on (Sections 4.4.4 and 4.4.5), and porting the non-stationary dynamics into the NS-3 backend so packet-level results can confirm the mock findings. On the system side: replacing the TCP-like subflows with a real Multipath QUIC transport, and enriching the VMAF surrogate so quality responds to delivery conditions, not just bitrate. On the research side, three directions stand out: *inferring* intent from traffic rather than assuming it is declared; *online and safe* learning against live

traffic with bounded worst-case behavior; and pushing the "one principle, every controllable decision" recipe further – for instance, conditioning the multipath agents on an explicit intent vector as the single-path selector does, unifying the two systems into a single intent-conditioned, cross-layer controller for path-aware networks.

Bibliography

[1] S. Abbasloo, C.-Y. Yen, and H. J. Chao. Classic meets modern: A pragmatic learning-based congestion control for the Internet. In *Proceedings of the ACM SIGCOMM Conference*, pages 632–647, 2020.

[2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[3] O. Bourak, T. Burman, T. John, H. Daltrophe, T. Shrot, and D. Hausheer. Path optimization using DQN in the SCION Internet architecture. In *Cyber Security, Cryptology, and Machine Learning (CSCML 2025)*, volume 16244 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.

[4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura. Intent-based networking – concepts and definitions. RFC 9315, Internet Engineering Task Force (IETF), 2022.

[5] Q. D. Coninck and O. Bonaventure. Multipath QUIC: Design and evaluation. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, pages 160–166, 2017.

[6] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch. TCP extensions for multipath operation with multiple addresses. RFC 8684, Internet Engineering Task Force (IETF), 2020.

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.

[8] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, Internet Engineering Task Force (IETF), 2021.

[9] N. Jay, N. H. Rotman, B. Godfrey, M. Schapira, and A. Tamar. A deep reinforcement learning perspective on Internet congestion control. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3050–3059, 2019.

[10] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara. Toward a practical perceptual video quality metric. Netflix Technology Blog, 2016. `https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652`.

[11] H. Mao, R. Netravali, and M. Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the ACM SIGCOMM Conference*, pages 197–210, 2017.

[12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[13] A. Perrig, P. Szalachowski, R. M. Reischuk, and L. Chuat. *SCION: A Secure Internet Architecture*. Springer, 2017.

[14] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1312–1320, 2015.

[15] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[16] Y. Shen, R. Chen, B. Wang, J. Chen, H. Zhang, M. Wang, Y. Liu, M. Xu, and Z. Meng. Mortise: Auto-tuning congestion control to optimize QoE via network-aware parameter optimization. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026.

[17] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar. Learning to route. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 185–191, 2017.

[18] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, pages 2094–2100, 2016.

[19] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.

[20] H. Wu, O. Alay, A. Brunstrom, S. Ferlin, and G. Caso. Peekaboo: Learning-based multipath scheduling for dynamic heterogeneous environments. *IEEE Journal on Selected Areas in Communications*, 38(10):2295–2310, 2020.

[21] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang. Experience-driven networking: A deep reinforcement learning based approach. In *IEEE INFOCOM 2018 – IEEE Conference on Computer Communications*, pages 1871–1879, 2018.

[22] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[23] H. Zhang, W. Li, S. Gao, X. Wang, and B. Ye. ReLeS: A neural adaptive multipath scheduler based on deep reinforcement learning. In *IEEE INFOCOM 2019 – IEEE Conference on Computer Communications*, pages 1648–1656, 2019.

A Reproducibility Details

This appendix collects the hyperparameters, reward-profile definitions, and supplementary results that support the main text. Values still to be finalized from the experiment runs are marked.

A.1 Single-Path Selection: Reward Profiles

The conditional path-selection agent is trained and evaluated across the reward-weight vectors $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ of Eq. (4.2), spanning distinct application intents. Table A.1 lists the six profiles used, together with the label each carries in Chapter 4.

Table A.1: The six reward profiles (application intents) used in the single-path selection study (Chapter 4). w_1 weights goodput, w_2 trust, w_3 loss and w_4 delay within the trust term, and w_{probe} the probing penalty. All six are used in training on a stratified schedule; the four marked with a chapter label are also used in the evaluation of §4.4.

Profile	Label in Chapter 4	w_1	w_2	w_3	w_4	w_{probe}
`bandwidth_max`	Throughput	1.00	0.00	0.05	0.05	0.05
`delay_averse`	Low-Latency	0.10	0.90	0.15	1.00	0.05
`loss_averse`	Low-Loss	0.05	0.95	1.00	0.15	0.05
`balanced_extreme`	Balanced	0.55	0.45	0.55	0.55	0.05
`probe_minimal`	–	0.70	0.30	0.50	0.50	0.001
`probe_averse`	–	0.70	0.30	0.50	0.50	0.35

A.2 Single-Path Selection: Agent Hyperparameters

Table A.2 lists the hyperparameters shared by the flat, scoring, and conditional agents of §4.3.1; the flat agent differs only in training for 533 episodes rather than 666.

Two caveats on this table. The ε schedule is stated as its configured range, but a run is too short to traverse it: at 0.995 decay per episode, ε reaches only 0.036 after 666 episodes and 0.069 after 533, so the 0.01 floor is never attained and all reported agents retain a few percent of exploration at the end of training. Seeding

Table A.2: Hyperparameters for the single-path selection agents (Chapter 4).

Parameter	Value
Hidden layers / width	2 / 128
Learning rate (Adam)	10^{-3}
Discount γ	0.95
Batch size	32
Replay capacity	10^4 (minimum 500 before learning)
Target update	soft (Polyak), $\tau = 0.05$, every 100 gradient steps
Prioritized replay	$\alpha = 0.6$, β annealed $0.4 \rightarrow 1.0$
Exploration	ε-greedy, $1.0 \rightarrow 0.01$, decay 0.995/episode
Dropout	0.1
Gradient clipping	1.0
Episodes × steps	666×24 (flat: 533×24)
Gradient step frequency	every 4 environment steps
Invalid-action mask value	-10^9
Random seeds	5 (all agents)
Wall-clock per agent	1–3 min (22-core CPU, no GPU)

fixes network initialization, prioritized-replay sampling, and ε-greedy exploration; the environment's pair, hour, and profile streams are deliberately left fixed across seeds, so every seed sees the identical stream of training contexts and only the learning process varies.

A.3 Multipath System: QoE Weights and SAC Hyperparameters

The QoE reward of Eq. (5.1) uses default weights $(a, b, c, d) = (1, 0.5, 0.5, 1)$ with latency and jitter normalizers of 200 ms and 50 ms; the application agent acts every 1 s and the transport agent every frame (≈33 ms at 30 fps) over a bitrate range of 300 – 6000 kbit/s. **[TODO:** *tabulate the SAC hyperparameters (network sizes, learning rates, discount, target entropy, temperature schedule, replay capacity, batch size) and the exact topology parameters (per-path capacity, RTT, cross-traffic intensity) for the static four-path and dynamic six-path scenarios.***]**

A.4 Supplementary Results

Figure A.1 substantiates the convergence discussion of §4.3.1. The per-episode statistics also carry the stratified schedule's profile label for each episode, giving roughly 111 episodes per intent for the conditional agents; these per-intent curves

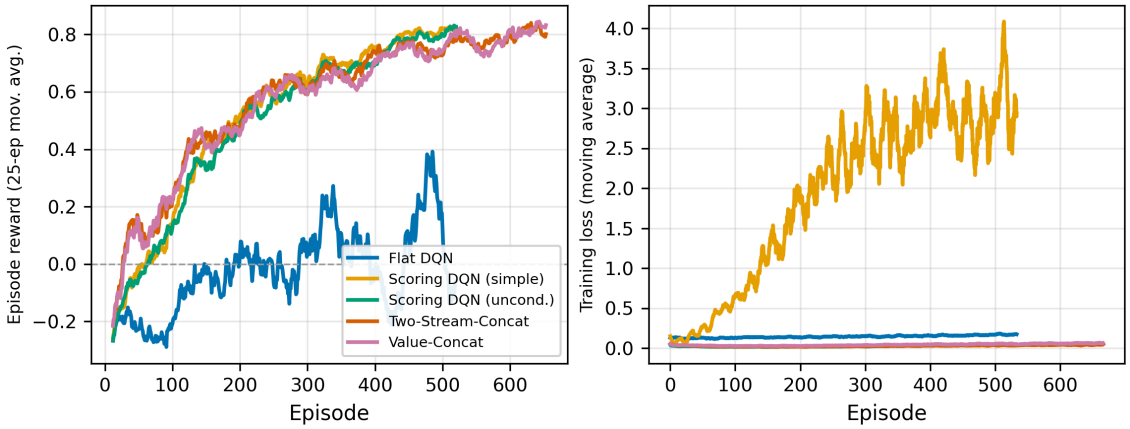

Figure A.1: Training curves for the single-path agents of §4.3.1: episode reward (25-episode moving average, left) and training loss (right) over the 533 or 666 episodes each agent was trained for. The four path-scoring agents converge to within 0.01 of one another at roughly +0.81, while the flat DQN plateaus near +0.05 with a *rising* loss despite the identical budget, data, and optimizer – the evidence that its deficit is architectural rather than a matter of training time. The simple scoring agent's loss also diverges, which is why the enhanced variant is used throughout.

are the only place the two probe-frugality profiles of Table A.1 appear, since §4.4.1 excludes them from the conditioning studies.

[TODO: *two further sets of supplementary plots. (i) Per-intent path-choice distributions for the single-path selector. (ii) Representative within-episode split traces for the multipath system.*]